

Retrieval Augmented Generation

Components and Construction of the RAG Architecture

How this tutorial is organized

01



Part One: Chapters 1 to 3

Foundations, the motivation for RAG, the formal model, its history, and the two pipelines that every RAG system is built from.

02



Part One: Chapters 4 to 7

RAG paradigms from naive to agentic, systematic failure analysis, evaluation methodology, and production concerns such as security and observability.

03



Part Two: Components 1 to 6

Data sources, parsing and cleaning, chunking, embedding models, vector stores and indexes, and query transformation.

04



Part Two: Components 7 to 11

Retrieval, reranking, prompt assembly, the generator LLM, and guardrails, caching and orchestration, ending with a blueprint.

LEARNING OBJECTIVES

What you will be able to do after this tutorial

1

Explain why RAG exists

Articulate the limits of parametric knowledge and how retrieval addresses hallucination, staleness and missing provenance.

3

Make principled design choices

Select chunking, embedding, indexing, retrieval and reranking strategies based on corpus, query and latency constraints.

5

Evaluate rigorously

Build a golden dataset and measure recall, ranking quality, faithfulness and answer relevance.

2

Draw the architecture from memory

Name every component of the indexing and query pipelines and the data contract between each pair of stages.

4

Diagnose failures

Separate retrieval failures from generation failures and map each symptom to its root cause and remedy.

6

Ship safely

Apply access control, prompt injection defences, observability and cost controls in production.

01

Foundations of RAG

Why large language models need external knowledge, what RAG is formally, and how the idea evolved from classical information retrieval to modern neural pipelines.

Parametric limits

Definition

Formal model

RAG Sequence and RAG Token

History

The limits of parametric knowledge

A language model knows only what it absorbed during training, frozen at a cutoff, compressed into weights, and impossible to cite.

Large language models are extraordinary pattern completers, but their knowledge is parametric: distributed across billions of weights, fixed at the moment training ended, blended without sources, and blind to anything private to your organisation. Retrieval Augmented Generation exists to close that gap.

MOTIVATION

Four failure modes of a standalone LLM



Hallucination

When the answer is not well represented in the weights, the model still produces fluent text, and fluency is easily mistaken for truth.



Staleness

Knowledge stops at the training cutoff. Prices, policies, versions and events after that date are unknown or wrong.



No private data

Internal wikis, contracts, tickets and code were never in the training set, so the model cannot know them.



No provenance

The model cannot point to where a claim came from, which blocks auditing, compliance and user trust.

TWO KINDS OF MEMORY

Parametric versus nonparametric memory

Dimension	Parametric memory (model weights)	Nonparametric memory (retrieval index)
Where knowledge lives	Distributed across billions of weights	Explicit documents, chunks and vectors in a store
How it is updated	Retraining or fine tuning, slow and costly	Insert, update or delete a document, near instant
Inspectability	Opaque; no way to list what is known	Fully inspectable; every item can be read
Attribution	Not possible	Every passage has a source, page and timestamp
Access control	Everything is visible to every user	Filter by user permissions at query time
Strength	Reasoning, language, general world knowledge	Precise, current, specific and private facts

DEFINITION

What RAG is, precisely

Retrieval Augmented Generation is an architecture that, for each input, retrieves relevant evidence from an external corpus and conditions a generative model on that evidence to produce a grounded output.

Three ideas are packed into the definition. Retrieval selects a small evidence set from a large corpus. Augmentation inserts that evidence into the model input. Generation produces the answer conditioned on both the question and the evidence, ideally with citations back to the sources.

INTUITION

The open book exam analogy

The most useful intuition for RAG is the difference between a closed book exam and an open book exam.

In a closed book exam, the student answers from memory. A brilliant student writes fluent answers, but when memory fails, confident guesses appear. That is a standalone LLM.

In an open book exam, the student first finds the relevant pages, reads them, and then writes an answer that quotes them. The quality now depends on two skills: finding the right pages and reading them faithfully.

Those two skills map exactly onto the two halves of RAG: the retriever and the generator.

MAPPING THE ANALOGY

The library → the document corpus

The index cards → the search index

Finding the right pages → retrieval

Choosing the best pages → reranking

Laying pages on the desk → prompt assembly

Writing the answer → generation

Quoting page numbers → citations

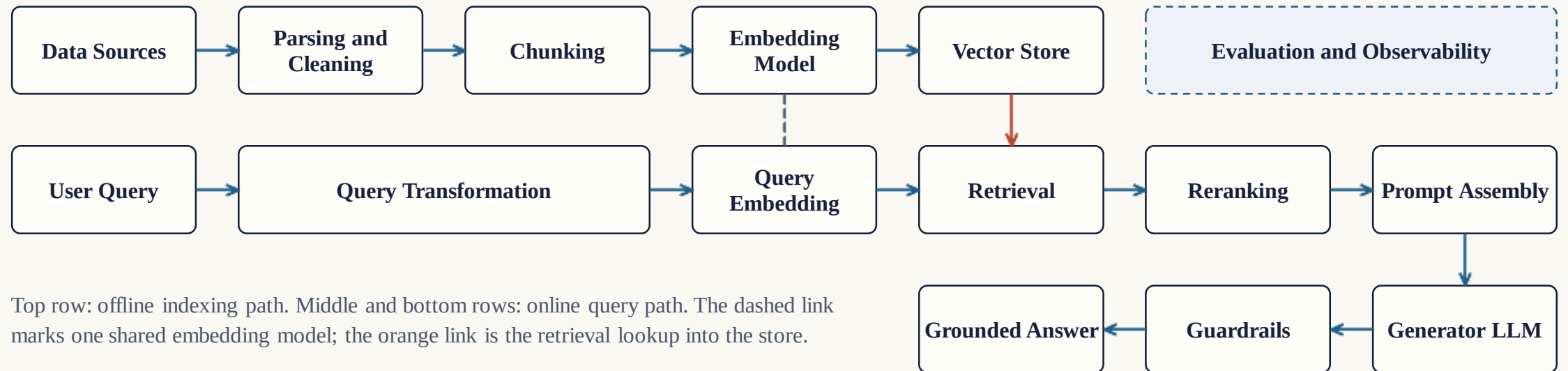
A wrong page on the desk → a retrieval failure

Misreading a correct page → a generation failure

THE BIG PICTURE

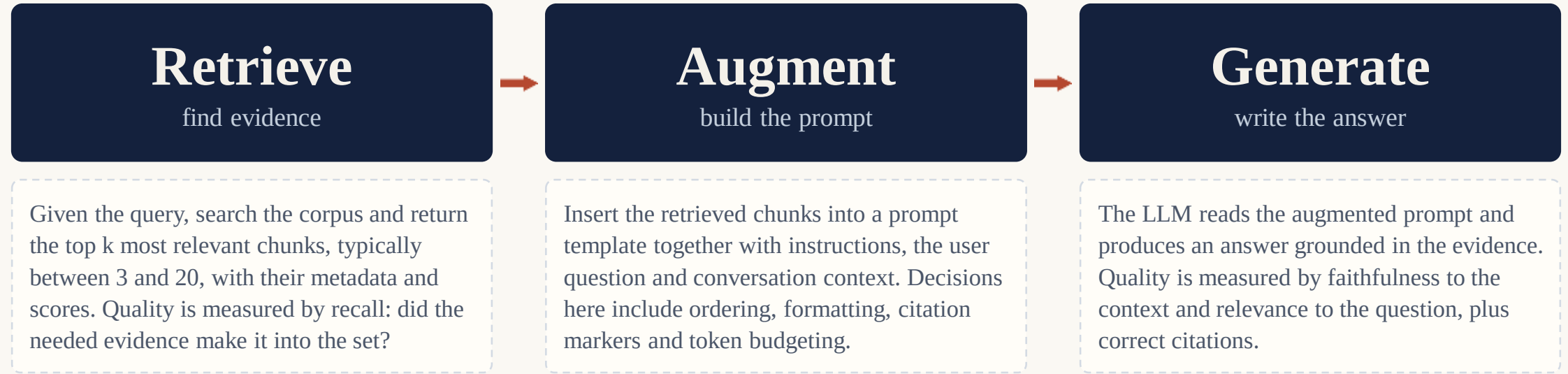
RAG in one diagram

The canonical RAG architecture has an offline indexing path that prepares knowledge and an online query path that uses it. Every box below is a component analyzed in Part Two.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

Retrieve, augment, generate



RAG as latent variable marginalization

$$p(y \mid x) \approx \sum \text{over } z \text{ in top } k \text{ of } p_{\eta}(z \mid x) \cdot p_{\theta}(y \mid x, z)$$

The retrieved document is treated as a latent variable and marginalised out (Lewis et al., 2020)

x

The input: the user question plus any conversation context.

z

A latent retrieved document or chunk drawn from the corpus.

y

The generated output sequence, the answer.

$p_{\eta}(z \mid x)$

The retriever: a distribution over documents given the query, parameterised by η .

$p_{\theta}(y \mid x, z)$

The generator: the probability of the answer given query and evidence, parameterised by θ .

Top k marginalisation

Summing over all documents is intractable, so the sum is approximated over the k highest scoring ones.

RAG Sequence versus RAG Token

Aspect	RAG Sequence	RAG Token
Core idea	One retrieved document conditions the whole answer	Each generated token may rely on a different document
Marginalisation	Over documents, once per complete output sequence	Over documents, separately at every token step
Formula sketch	$\sum \text{over } z \text{ of } p(z x) \cdot \prod \text{over } i \text{ of } p(y \text{ at } i \mid x, z, \text{ earlier tokens})$	$\prod \text{over } i \text{ of } \sum \text{over } z \text{ of } p(z x) \cdot p(y \text{ at } i \mid x, z, \text{ earlier tokens})$
Best suited to	Answers supported mainly by a single source	Answers that synthesise facts from several sources
Modern analogue	Answer from the single best chunk	Put many chunks in context and let attention mix them

From term weighting to neural embeddings

1972	1994	2013	2017	2018	2019
Inverse document frequency	Okapi BM25	word2vec	Transformer and FAISS	BERT	Sentence BERT
Karen Spärck Jones formalises IDF: rare terms are more informative than common ones.	Robertson and colleagues introduce BM25, still the lexical baseline to beat three decades later.	Mikolov and colleagues show words can be embedded as dense vectors that capture meaning.	Attention is All You Need introduces the Transformer; Facebook releases FAISS for fast vector search.	Bidirectional encoders produce contextual embeddings and transform search and understanding.	Siamese BERT networks produce sentence embeddings that are comparable by cosine similarity.

The modern RAG era, 2020 onwards

2020	2021	2022	2023	2024	Today
REALM, RAG, DPR, FiD	RETRO	HyDE and Atlas	Self RAG, Lost in the Middle	GraphRAG, RAPTOR, CRAG	Agentic RAG
Retrieval enters pretraining (REALM), the RAG paper names the pattern, DPR shows dense retrieval beats BM25 on open QA.	DeepMind retrieves from a trillion token database during language modelling.	Hypothetical document embeddings improve zero shot retrieval; Atlas shows few shot learning with retrieval.	Models learn to decide when to retrieve and to critique evidence; long contexts are shown to underuse middle positions.	Knowledge graphs, recursive summary trees and corrective retrieval evaluators extend the pattern.	Retrieval becomes a tool called by agents that plan, search iteratively and verify.

02

Why RAG, and When

The benefits RAG delivers, how it compares with fine tuning and prompting, how it relates to long context models, and when it is the wrong tool.

Benefits

RAG versus fine tuning

Decision matrix

Anti patterns

Long context

Use cases

Six benefits of the RAG architecture



Grounded accuracy

Answers are anchored in retrieved evidence, which reduces unsupported claims when retrieval is good.



Freshness

Update knowledge by reindexing documents; no retraining is needed to reflect today's policy.



Provenance

Every answer can cite the chunks it used, which enables audit and user verification.



Access control

Filtering at retrieval time means users only see evidence they are permitted to see.



Cost efficiency

A smaller model with good retrieval can match a larger model on knowledge intensive tasks.



Modularity

Retriever, reranker and generator can each be swapped and upgraded independently.

COMPARISON

RAG versus fine tuning versus prompting

Criterion	Prompt engineering	RAG	Fine tuning
Adds new knowledge	Only what fits in the prompt	Yes, at scale, updatable	Partially, becomes stale
Changes behaviour or style	Somewhat	Weakly	Strongly
Citations and provenance	No	Yes	No
Update cost	Very low	Low: reindex documents	High: retrain and reevaluate
Hallucination control	Weak	Strong when retrieval is good	Moderate
Upfront engineering	Minimal	Moderate to high	High: data, compute, evaluation
Typical role	Starting point	Knowledge layer	Behaviour and format layer

DECISION MATRIX

Choosing the right adaptation strategy

↑ **Need to change model behaviour, tone or format**

Fine tuning

Stable knowledge, but a new format, style, or task skill is needed. Example: domain specific report formatting.

RAG plus fine tuning

Specific, changing knowledge and a specialised behaviour. Example: a clinical assistant that must cite guidance in a set format.

Prompt engineering

General knowledge and default behaviour suffice. Example: summarising pasted text.

RAG

Knowledge intensive, changing or private content with standard behaviour. Example: an internal policy assistant.

Need for fresh, specific or private knowledge →

When RAG is the wrong tool



Pure reasoning tasks

Maths, logic puzzles and code generation from a spec rarely benefit from document retrieval; tools or reasoning models fit better.



Tiny, static knowledge

If all the knowledge fits comfortably in the prompt and rarely changes, put it in the system prompt instead.



Aggregate analytics

Questions like total revenue by region need SQL over structured data, not semantic search over text chunks.



Whole corpus synthesis

Questions such as the main themes across all documents are poorly served by top k retrieval; use summaries or GraphRAG.

A COMMON QUESTION

RAG and long context windows

Why long context does not replace RAG

- Cost and latency grow with prompt length; sending an entire corpus on every query is wasteful.
- Corpora of millions of documents far exceed any context window.
- Models underuse information buried in the middle of long inputs.
- Access control and provenance still require selecting specific documents.

How long context strengthens RAG

- Larger windows let you pass more, and larger, retrieved chunks.
- Parent document and small to big retrieval become practical.
- Retrieval recall can be traded against a bigger context budget.
- Whole document reading after retrieval enables deeper synthesis.

APPLICATIONS

Typical use cases and where they break

Use case	Typical corpus	Query style	Critical risk
Customer support assistant	Help centre articles, manuals, past tickets	Short, informal, typo prone	Outdated or conflicting articles
Enterprise knowledge search	Wikis, policies, slide decks, email	Vague, jargon heavy	Permission leakage across teams
Legal and compliance review	Contracts, regulations, case law	Precise, clause level	Missing a qualifying clause
Clinical and scientific QA	Guidelines, papers, trial reports	Technical, multihop	Unsupported medical claims
Developer documentation	API references, code, changelogs	Version specific	Answering for the wrong version
Financial research	Filings, earnings calls, tables	Numeric, comparative	Table parsing and number errors

03

The Two Pipelines

Every RAG system is two pipelines joined by an index: an offline indexing pipeline that prepares knowledge and an online query pipeline that consumes it.

Indexing pipeline

Query pipeline

Worked traces

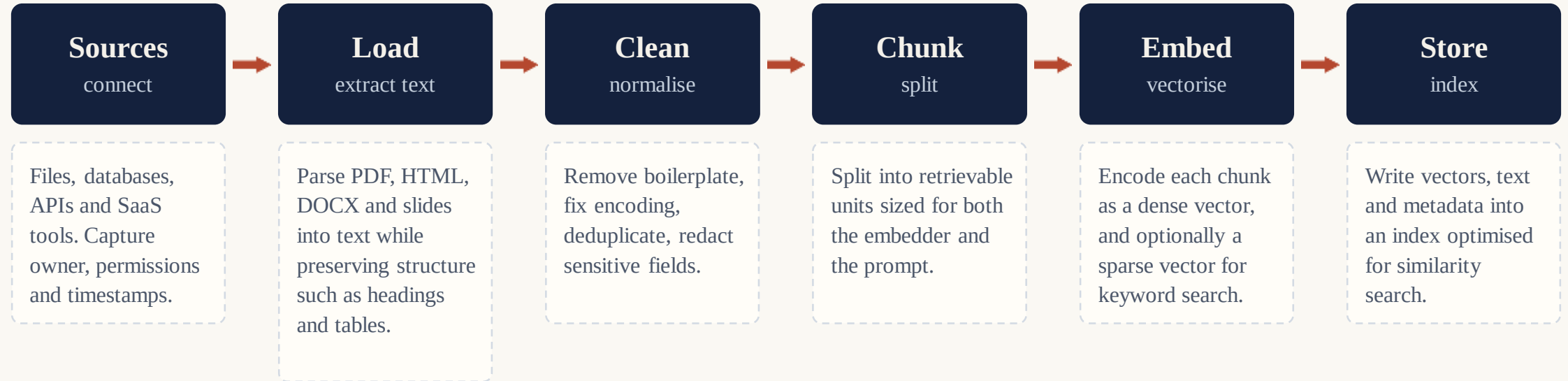
Data contracts

Latency budget

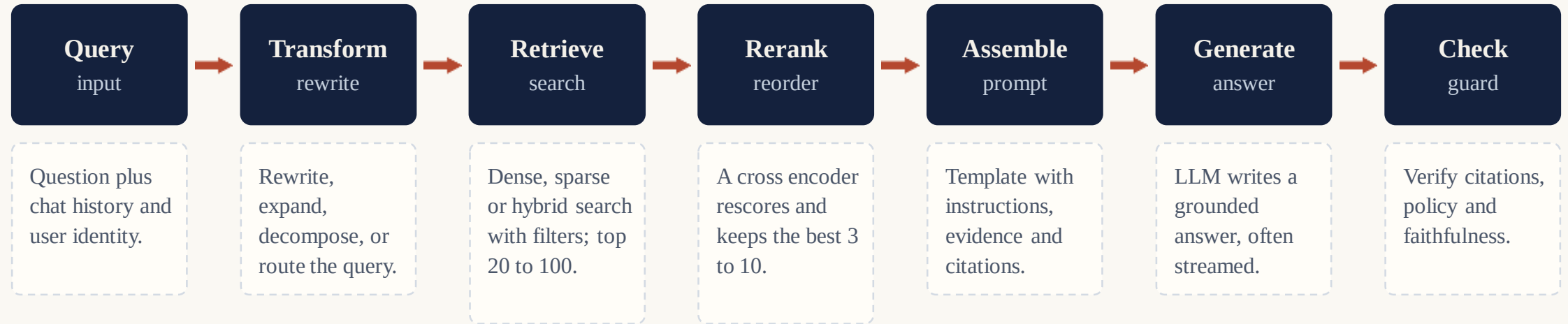
Token economics

OFFLINE PATH

The indexing pipeline



The query pipeline



WORKED TRACE

The life of a query

1

User asks

“What is our parental leave policy for contractors in the UK office?”

3

Hybrid retrieval

BM25 matches the exact terms “parental leave” and “contractors”; dense search finds a chunk titled “Family leave for non employees”. 50 candidates return.

5

Assemble and generate

Five chunks with source tags are placed in the prompt; the LLM answers and cites [HR Policy 4.2] and [Contractor Handbook 7].

2

Condense and rewrite

History is folded in; the query becomes “parental leave policy for contractors, United Kingdom”, and a filter region equals UK is extracted.

4

Rerank

A cross encoder scores each candidate against the question; the top 5 are kept, the family leave chunk ranked first.

6

Verify and log

Citations are checked against the retrieved set, the answer is streamed, and the full trace is logged for evaluation.

WORKED TRACE

The life of a document

1

Arrival

A 40 page PDF, “Contractor Handbook v7”, lands in a SharePoint folder; a change event triggers ingestion.

3

Enrichment

Metadata is attached: title, version 7, department HR, region UK, access group all staff, modified date.

5

Embedding

Each chunk is encoded as a 768 dimensional vector; a sparse BM25 representation is also built.

2

Parsing

A layout aware parser extracts text in reading order, detects 3 tables and 12 headings, and discards page headers and footers.

4

Chunking

Structure aware splitting produces 85 chunks of about 400 tokens, each prefixed with its section path such as “7 Leave > 7.3 Family leave”.

6

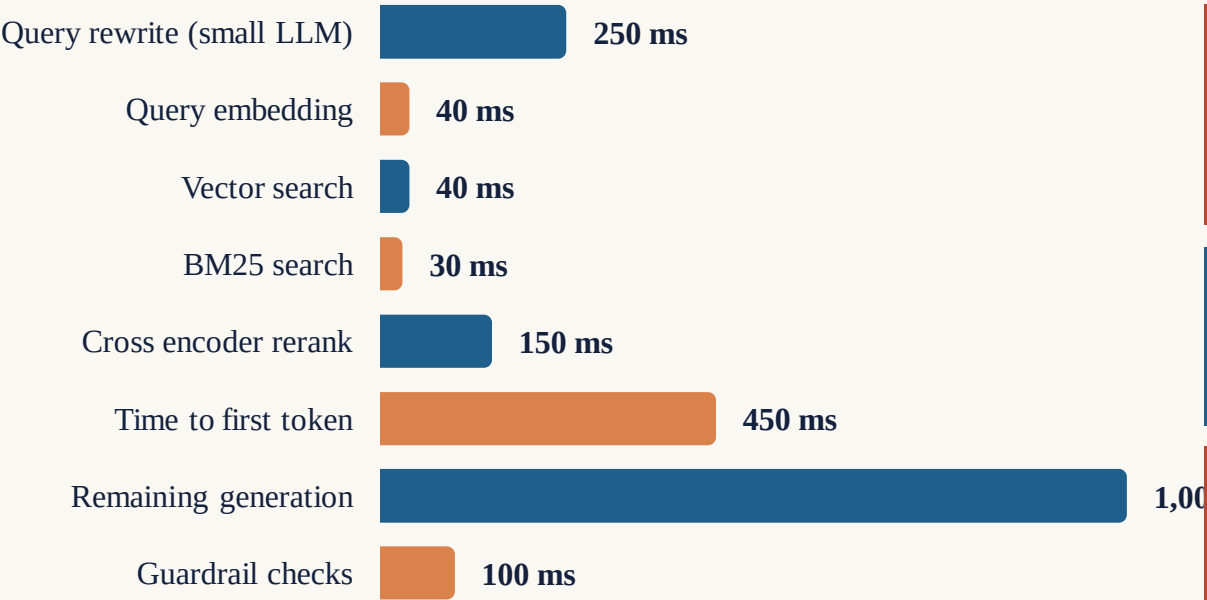
Indexing

Vectors and metadata are upserted; the six chunks from version 6 are marked superseded and removed from the live index.

Data contracts between stages

Stage	Consumes	Produces	Metadata that must survive
Loader	Raw files and records	Document objects with text	Source URI, owner, ACL, timestamps
Parser and cleaner	Documents	Structured text elements	Page numbers, headings, element types
Chunker	Structured text	Chunks with offsets	Parent document ID, section path, position
Embedder	Chunk text	Vectors	Model name and version, dimension
Vector store	Vectors plus metadata	Searchable index	All of the above, filterable
Retriever	Query and filters	Scored candidates	Scores, ranks, retrieval method
Prompt builder	Ranked chunks	Prompt text	Citation IDs mapped to sources

Typical Structure of a latency budget



Example latency budget per stage, in milliseconds. Illustrative design targets.

How to read this

An example design budget for a 2 second first answer target, not a benchmark. Your numbers depend on hardware, models and load.

Where the time goes

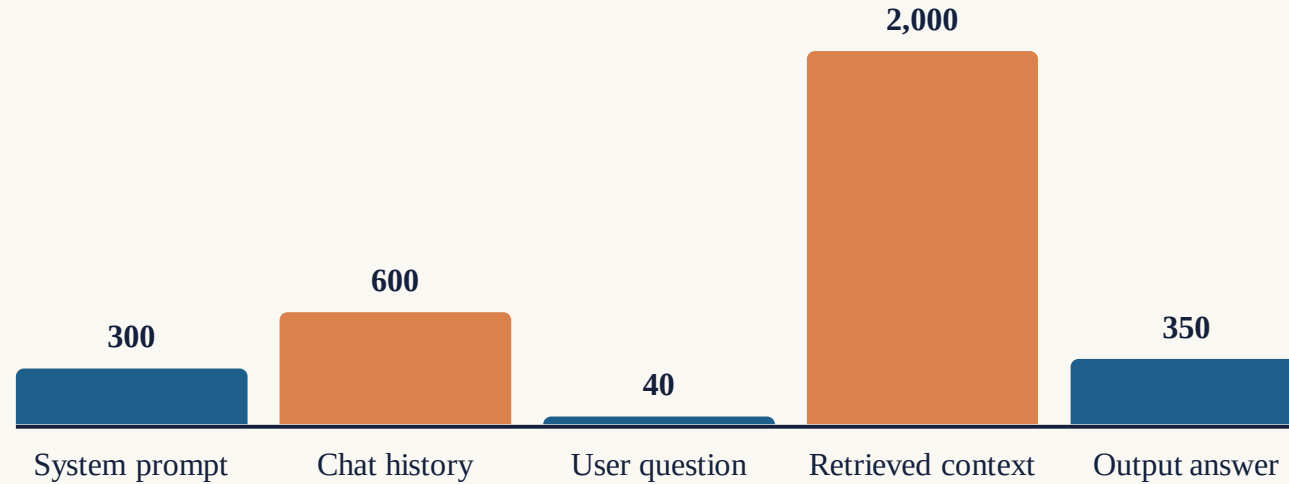
Generation dominates. Streaming hides most of it, so time to first token is the user facing metric.

Parallelism

Vector search and BM25 run concurrently, so only the slower of the two sits on the critical path.

COST

Where the tokens go



Token composition of one example request (tokens). Assumptions: k equals 5, chunks of 400 tokens.

The arithmetic

Five chunks of 400 tokens make 2,000 context tokens. Input totals 2,940 tokens and output 350, so context is about 68 percent of the input.

The lever

Halving k or chunk size halves the dominant cost. Reranking lets you keep fewer, better chunks.

Caching

Stable prefixes such as the system prompt can be cached by many providers to reduce repeated cost.

04

RAG Paradigms

How the architecture has evolved from a simple retrieve then read pipeline into advanced, modular, graph based, self reflective and agentic systems.

Naive RAG

Advanced RAG

Modular RAG

Agentic RAG

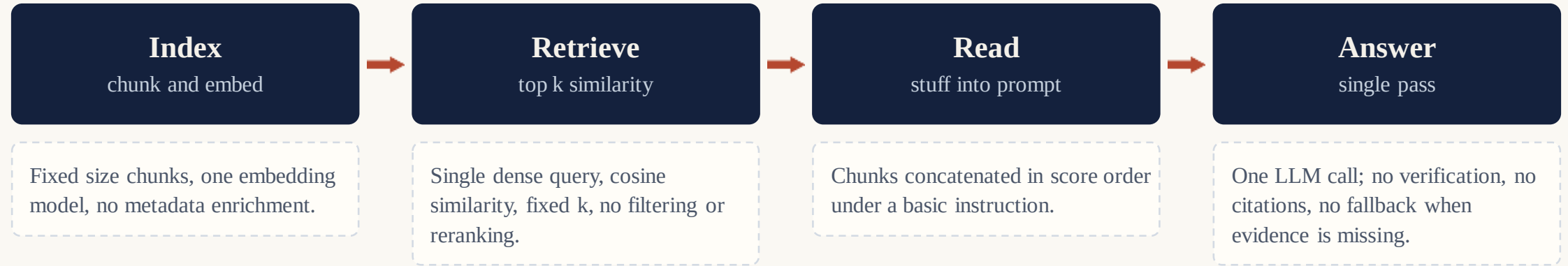
GraphRAG

Self RAG and CRAG

Multimodal RAG

PARADIGM ONE

Naive RAG: retrieve, then read



Advanced RAG: optimize before and after retrieval

Pre retrieval optimizations

- Better indexing: structure aware chunking, metadata enrichment, chunk summaries.
- Query rewriting and expansion to close the vocabulary gap.
- Query decomposition for multihop questions.
- Routing queries to the right index or data source.

Post retrieval optimizations

- Reranking candidates with a cross encoder.
- Context compression to remove irrelevant sentences.
- Deduplication and diversity through maximal marginal relevance.
- Ordering chunks to counter position bias in long prompts.

PARADIGM THREE

Modular RAG: composable building blocks

Search module

Queries many sources: vector stores, search engines, SQL, graphs, APIs.

Memory module

Uses conversation history and the model's own prior outputs as retrievable memory.

Fusion module

Issues several query variants and merges results with reciprocal rank fusion.

Routing module

Classifies a query and sends it to the best pipeline or index.

Predict module

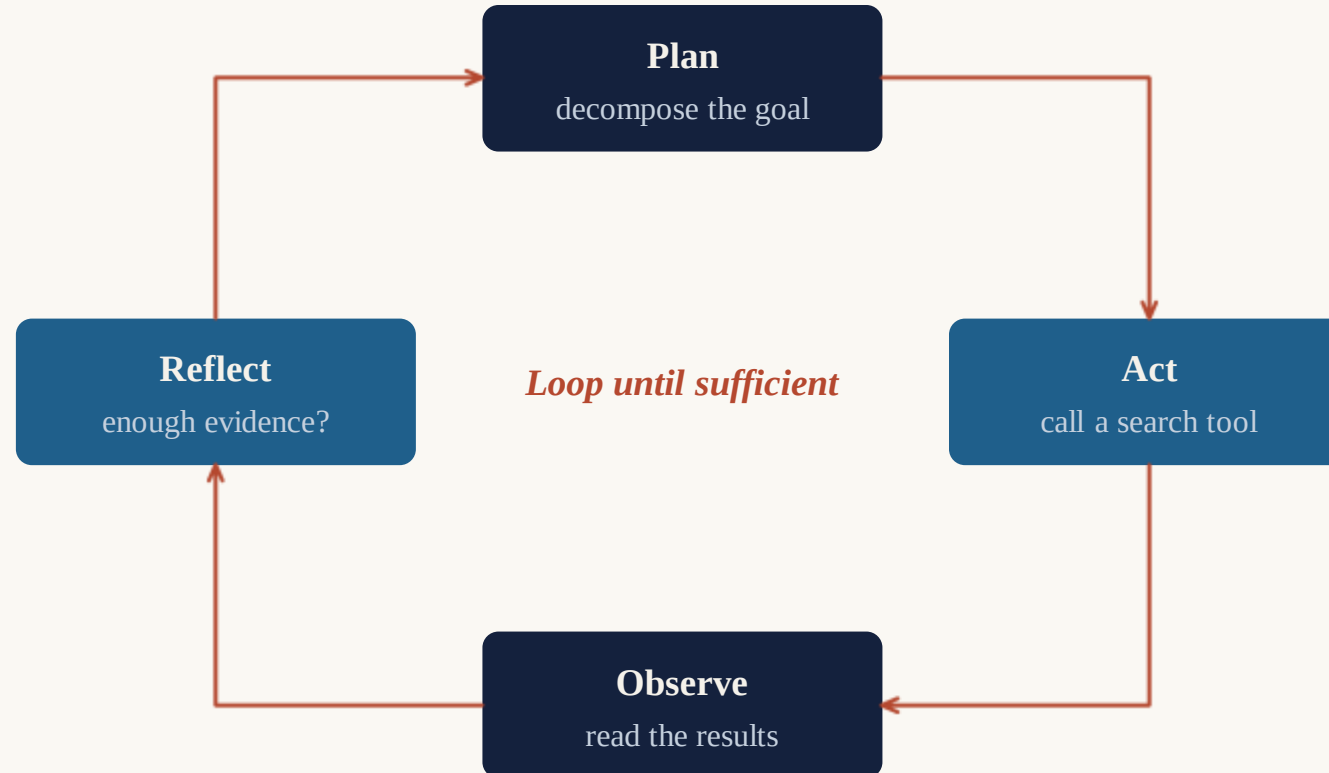
Asks the LLM to generate context, such as a hypothetical answer, before retrieving.

Task adapter

Adapts retrieval and prompts to the downstream task: QA, summarisation, extraction.

PARADIGM FOUR

Agentic RAG: retrieval inside a reasoning loop



Tools

Vector search, web search, SQL, calculators and APIs are all callable tools with descriptions.

Stopping rule

The agent answers when evidence suffices, or stops at a step or token budget.

Trade off

Much stronger on multihop questions, but slower, costlier and less predictable than a fixed pipeline.

PARADIGM FIVE

GraphRAG: retrieval over a knowledge graph

GraphRAG, popularised by Microsoft Research in 2024, uses an LLM to extract entities and relationships from the corpus into a knowledge graph.

The graph is partitioned into communities using a clustering algorithm such as Leiden, and a summary is written for each community at several levels.

Local search answers entity specific questions by traversing neighbourhoods. Global search answers whole corpus questions by map reducing over community summaries.

This directly addresses the whole corpus synthesis weakness of top k vector retrieval, at a substantial indexing cost.

INDEXING STAGES

1. Chunk the corpus into text units
2. LLM extracts entities, relations, claims
3. Build the graph; merge duplicate entities
4. Detect communities hierarchically
5. Summarise each community with an LLM
6. Query: local traversal or global map reduce

Strength: themes, relationships, multihop

Cost: many LLM calls during indexing

Self reflective and corrective RAG

Self RAG (Asai et al., 2023)

- The model is trained to emit reflection tokens.
- It decides whether retrieval is needed at all for a given input.
- It critiques each retrieved passage for relevance.
- It checks whether its own output is supported by the evidence.

Corrective RAG, CRAG (Yan et al., 2024)

- A lightweight evaluator grades retrieved documents.
- Correct: refine and use the documents.
- Incorrect: discard them and fall back to web search.
- Ambiguous: combine refined documents with web results.

Multimodal RAG



Text through captions

Convert images, charts and tables into text descriptions with a vision model, then index the text. Simple and compatible with any text retriever.



Joint embeddings

Use a model such as CLIP that embeds images and text in one shared space, enabling text to image and image to text retrieval.



Page image retrieval

Embed whole page screenshots with a vision language retriever, as in ColPali, avoiding brittle parsing of complex layouts.

SUMMARY

Comparing the paradigms

Paradigm	Control flow	Strength	Main cost	Choose when
Naive	Linear, one pass	Simple, fast	Low quality ceiling	Prototype or baseline
Advanced	Linear with extra stages	Precision and recall	More components	Most production systems
Modular	Composable flows	Flexibility	Design complexity	Varied query types
Agentic	Iterative loop	Multihop reasoning	Latency and cost	Complex research tasks
GraphRAG	Graph traversal	Global questions	Indexing cost	Relationship heavy data
Self or corrective	Conditional	Robustness	Grader calls	Noisy or partial corpora

05

How RAG Fails

A systematic catalogue of failure points, a diagnosis procedure that separates retrieval errors from generation errors, and the classic pathologies to watch for.

Seven failure points

Diagnosis tree

Lost in the middle

Chunking pathologies

Semantic gap

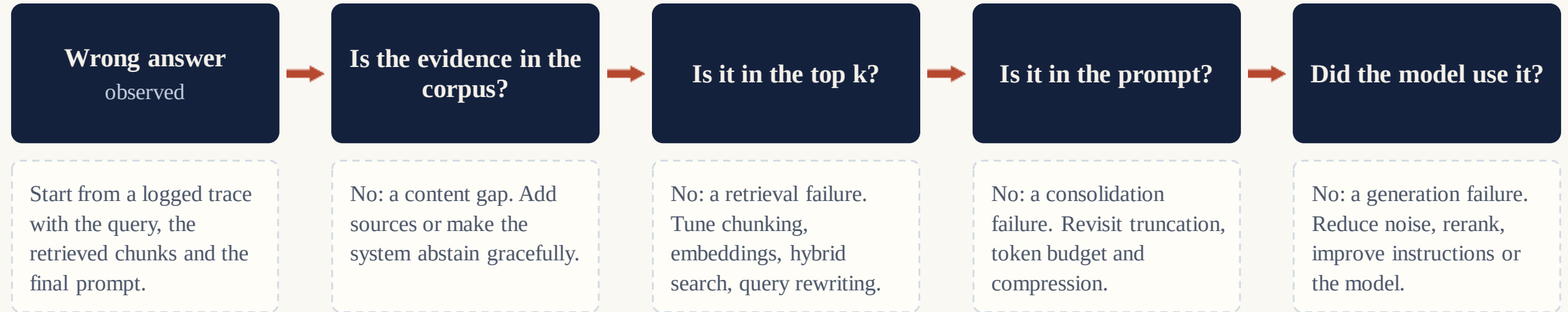
Unfaithful generation

The seven failure points

Failure point	What happens	Typical remedy
1. Missing content	The answer is not in the corpus at all	Coverage audits; teach the model to say it does not know
2. Missed top ranked	The answer exists but ranks below the cutoff k	Hybrid search, better embeddings, larger candidate pool
3. Not in context	Retrieved but dropped during consolidation	Token budgeting, compression, smarter truncation
4. Not extracted	In the prompt, but the model fails to use it	Less noise, reranking, better ordering and instructions
5. Wrong format	Ignores the requested table, list or schema	Structured output, examples, validators
6. Incorrect specificity	Too general or too detailed for the need	Query understanding and answer instructions
7. Incomplete	Answers part of a multipart question	Query decomposition and answer completeness checks

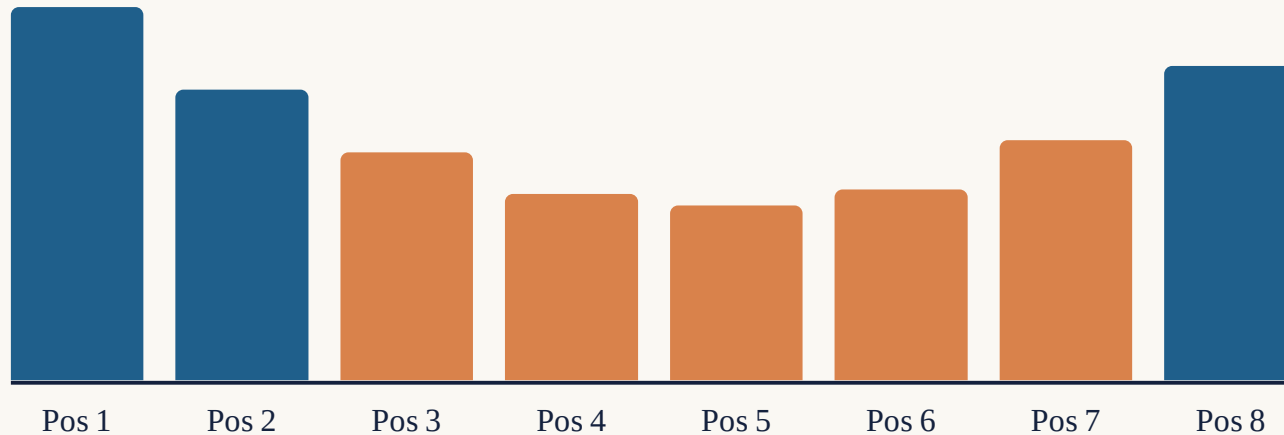
DEBUGGING PROCEDURE

Retrieval failure or generation failure?



PATHOLOGY

Lost in the middle



Schematic of the U shaped pattern: bar heights are illustrative, not measured values. Horizontal axis: position of the relevant passage.

The finding

Liu et al., 2023, observed that accuracy is highest when the relevant passage is at the start or end of a long context, and lower in the middle.

Implication

Where a chunk sits in the prompt affects whether it is used, not only whether it is present.

Mitigation

Put the strongest evidence first and last, keep k small after reranking, and compress noise.

Chunking pathologies

Split answer

The question and its answer fall into two different chunks, so neither chunk alone is sufficient.

Orphaned pronouns

A chunk says “it must be renewed annually” but the subject was named in the previous chunk.

Diluted embedding

A large chunk covering five topics produces a vector that is average and matches nothing well.

Broken tables

Rows are separated from their headers, turning a table into meaningless numbers.

Lost hierarchy

A clause appears without the section heading that qualifies it, such as “Exceptions”.

Boilerplate noise

Repeated headers, footers and disclaimers dominate chunks and pollute similarity.

PATHOLOGY

The semantic gap between questions and documents

Users and documents speak different languages. A user asks how to stop being charged; the policy document is titled Subscription cancellation procedure.

Questions are short, informal and underspecified. Documents are long, formal and declarative. Their embeddings may sit far apart even when one answers the other.

Lexical search fails when there is no word overlap; dense search can fail on exact identifiers, product codes and rare names.

Remedies include query rewriting, HyDE, hybrid retrieval, and indexing synthetic questions for each chunk.

SAME MEANING, DIFFERENT WORDS

User: “why was I charged twice”

Doc: “Duplicate billing and refunds”

User: “can I bring my dog”

Doc: “Pet policy for tenants”

User: “error E1043 on startup”

Doc: “E1043: licence server unreachable”

First two: lexical search struggles

Last one: dense search may struggle

Hallucination despite retrieval



Prior over evidence

When retrieved text contradicts what the model learned in training, the model may prefer its prior.



Conflicting sources

Two versions of a policy both appear in context; the model blends them into an answer matching neither.



Filling the gaps

Evidence covers half the question; the model invents the other half fluently instead of saying so.



Citation drift

The answer cites a real chunk, but the cited chunk does not support the specific claim.

06

Evaluating RAG

What to measure, how to compute it, how to build a trustworthy test set, and how to close the loop with online feedback.

Evaluation layers

Retrieval metrics

Worked example

Generation metrics

LLM as a judge

Golden datasets

Online evaluation

FRAMEWORK

The layers of RAG evaluation

Retrieval quality

Is the right evidence found and ranked highly?

← Recall at k, precision at k, MRR, nDCG, hit rate

Context quality

Is the assembled context relevant and sufficient?

← Context precision, context recall, noise ratio

Generation quality

Is the answer faithful and relevant?

← Faithfulness, answer relevance, citation accuracy

End to end correctness

Does the answer match the ground truth?

← Answer correctness, exact match, human rating

System quality

Is it fast, affordable, safe and reliable?

← Latency percentiles, cost per query, refusal and error rates

MEASUREMENT

Retrieval metrics

Metric	Definition	What it tells you
Hit rate at k	Share of queries with at least one relevant item in the top k	Coarse check that retrieval finds something useful
Recall at k	Relevant items in the top k divided by all relevant items	Coverage; the key metric for the first stage
Precision at k	Relevant items in the top k divided by k	Noise level passed downstream
MRR	Mean over queries of 1 divided by the rank of the first relevant item	How early the first good result appears
nDCG at k	DCG divided by ideal DCG, with gains discounted by 1 over log2(rank + 1)	Graded ranking quality across the whole list

WORKED EXAMPLE

Computing retrieval metrics for one query

Rank	Retrieved chunk	Relevant?	Discounted gain, $1 / \log_2(\text{rank} + 1)$
1	Chunk B	No	0
2	Chunk A	Yes	$1 / \log_2(3) = 0.631$
3	Chunk D	No	0
4	Chunk C	Yes	$1 / \log_2(5) = 0.431$
5	Chunk E	No	0

0.67

Recall at 5: two of three relevant chunks (A, C, F) found

0.40

Precision at 5: two of five results relevant

0.50

Reciprocal rank: first relevant at rank 2

0.50

nDCG at 5: 1.062 divided by ideal 2.131

Generation and context metrics



Faithfulness

Share of claims in the answer that can be inferred from the retrieved context. Detects hallucination.



Answer relevance

How directly the answer addresses the question, regardless of correctness.



Context precision

Whether relevant chunks are ranked above irrelevant ones in the context.



Context recall

Whether the context contains everything needed for the reference answer.



Answer correctness

Agreement with a reference answer, combining factual overlap and semantic similarity.



Citation accuracy

Whether each cited source actually supports the sentence that cites it.

LLM as a judge

Designing a reliable judge

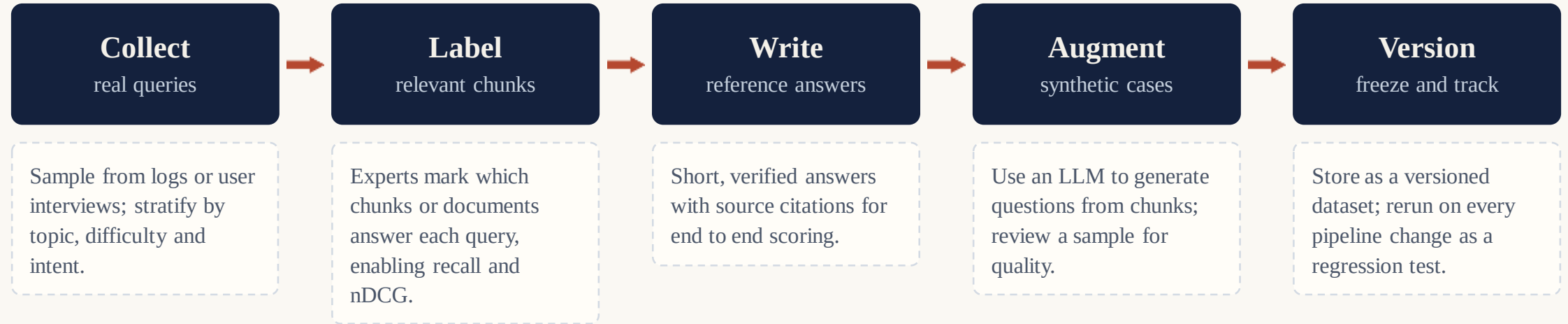
- Score one criterion at a time with a clear rubric.
- Ask for reasoning before the score, then parse the score.
- Use binary or small ordinal scales rather than 1 to 100.
- Calibrate against a few hundred human labelled examples.

Known biases to control

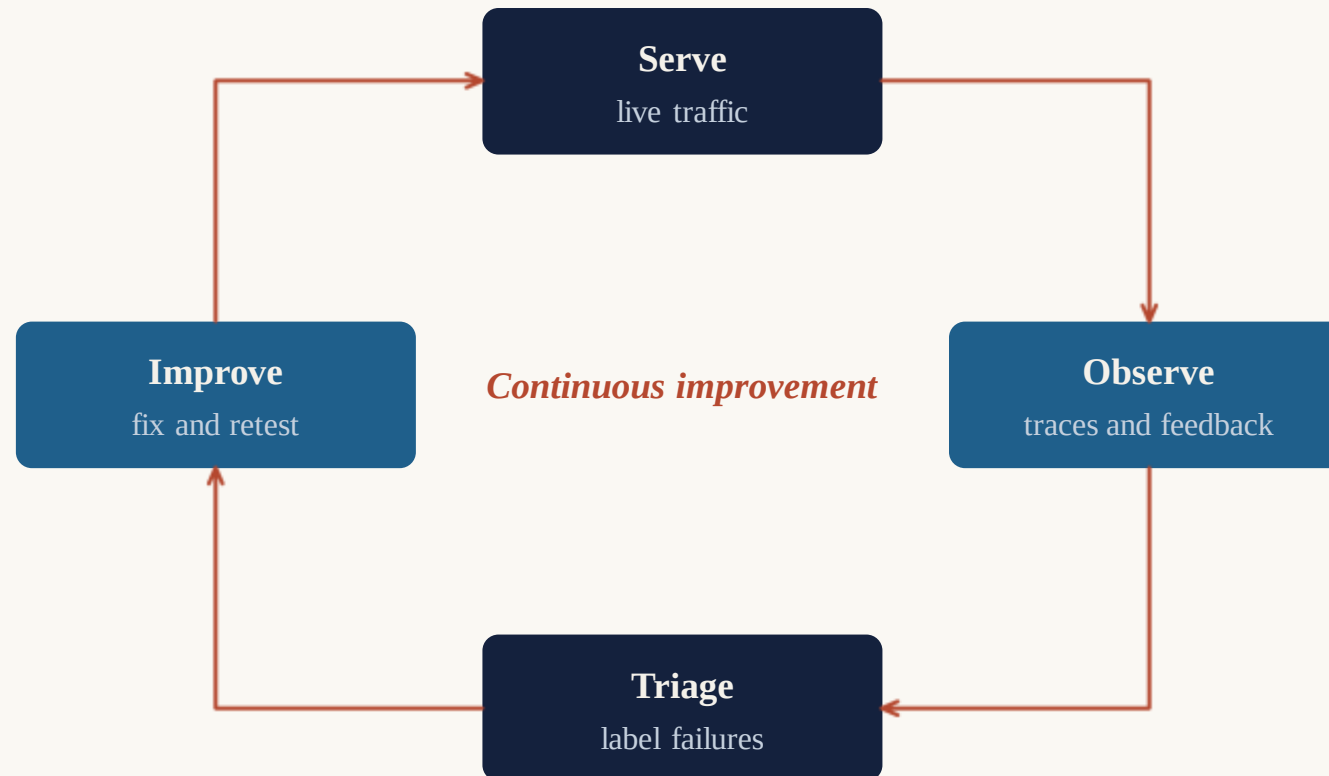
- Position bias: prefers the first option in pairwise comparisons.
- Verbosity bias: prefers longer answers.
- Self preference: may favour outputs from its own model family.
- Leniency: overlooks subtle unsupported claims without claim level checks.

MEASUREMENT

Building a golden evaluation dataset



Online evaluation and the feedback loop



Implicit signals

Copy events, follow up questions, reformulations and abandonment.

Explicit signals

Thumbs up or down, ratings, and free text comments.

Experiments

A/B tests on retrieval or prompt variants, judged by online metrics.

07

RAG in Production

Security, privacy, governance, observability, scaling and freshness: what changes when a prototype meets real users and real data.

Prompt injection

Access control

Governance

Observability

Scaling

Freshness

Security threats specific to RAG



Indirect prompt injection

Retrieved documents can contain instructions such as “ignore previous rules”. Treat retrieved text as data, delimit it, and never let it trigger privileged tools unchecked.



Permission leakage

Without document level access filtering, retrieval can surface content a user may not see. Enforce ACLs inside the retrieval query, not after generation.



Data poisoning

An attacker who can write to an indexed source can plant misleading content. Control ingestion sources and monitor for anomalies.

Privacy, governance and compliance

Data minimisation

Index only what the use case needs; exclude sensitive fields at ingestion.

PII handling

Detect and redact personal data before embedding, since vectors can leak content.

Right to erasure

Deletion must propagate to chunks, vectors, caches and logs under laws such as GDPR.

Retention

Apply the source system's retention rules to derived chunks and embeddings.

Lineage

Record which source, version and model produced every chunk and answer.

Human oversight

Define review paths for high stakes answers in legal, medical or financial settings.

Observability: what to log

Log item	Why it matters
Raw query, rewritten queries, extracted filters	Reveals whether query transformation helped or hurt
Candidate IDs, scores and ranks per retriever	Diagnoses retrieval failures and fusion behaviour
Reranker scores and the final selected chunks	Shows what the generator actually saw
Full prompt, token counts, model and parameters	Enables replay, cost tracking and regression analysis
Answer, citations, guardrail verdicts	Supports faithfulness checks and audits
Latency per stage, errors, user feedback	Drives performance tuning and continuous evaluation

Scaling and keeping the index fresh

Scaling

- Shard the index by tenant, time or hash as it grows beyond one node.
- Replicate for query throughput and availability.
- Quantise vectors to cut memory by four to thirty two times.
- Cache embeddings, retrieval results and answers.

Freshness

- Use change data capture or event hooks instead of full reloads.
- Hash content to reembed only what changed.
- Handle deletes and supersessions explicitly.
- Track an index lag metric: time from source edit to searchable.

Ten principles of effective RAG

01

Retrieval bounds quality

The generator cannot use evidence it never sees.

02

Start naive, measure

Build a baseline and a golden set first.

03

Parse carefully

Garbage text in means garbage answers out.

04

Chunk for meaning

Respect document structure and context.

05

Go hybrid

Combine lexical and dense retrieval.

06

Wide then narrow

Recall first, then rerank for precision.

07

Budget the context

Fewer, better chunks beat more noise.

08

Ground and cite

Instruct, verify and allow abstention.

09

Secure the index

Filter by permission, distrust retrieved text.

10

Trace everything

Every failure should be diagnosable.

PART TWO

II

Anatomy of a RAG System

One hundred slides that open every box in the architecture: what each component does, how it works internally, the design choices it offers, and how to choose between them.

Data sources

Parsing

Chunking

Embeddings

Vector stores

Query transformation

Retrieval

Reranking

Prompt assembly

Generator

Guardrails and orchestration

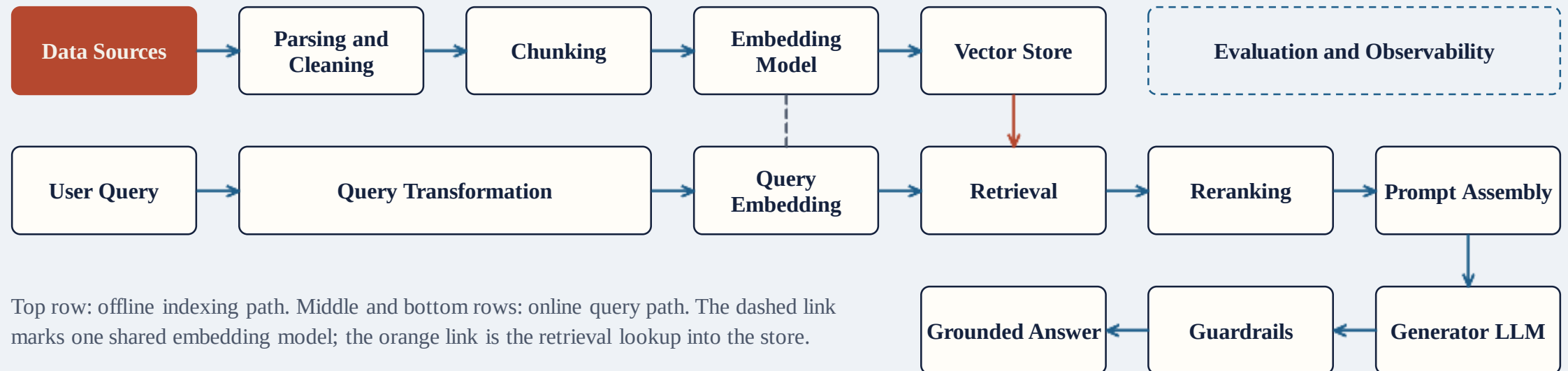
INVENTORY

The eleven components at a glance

#	Component	Primary responsibility	Key design decision
1	Data sources and ingestion	Connect to and extract content incrementally	Pull, push or change data capture
2	Parsing and cleaning	Turn files into clean, structured text	Layout aware versus plain extraction
3	Chunking	Split text into retrievable units	Size, overlap and boundary strategy
4	Embedding model	Map text to vectors that encode meaning	Model, dimension, domain fit
5	Vector store and index	Store and search vectors at scale	Index type, filtering, hosting
6	Query transformation	Make the query retrievable	Rewrite, expand, decompose, route
7	Retrieval	Find candidate evidence	Dense, sparse or hybrid; k
8	Reranking and filtering	Order and prune candidates	Cross encoder, MMR, compression
9	Prompt assembly	Build the augmented prompt	Template, order, budget, citations
10	Generator LLM	Write the grounded answer	Model, decoding, output format
11	Guardrails and orchestration	Verify, cache, coordinate, learn	Checks, caches, framework

Data Sources and Ingestion

Everything begins with where knowledge lives and how it reaches the pipeline: connectors, change detection, metadata and permissions captured at the source.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

COMPONENT 1

A taxonomy of knowledge sources

Source class	Examples	Extraction approach	Main challenge
Unstructured documents	PDF, Word, slides, scanned images	Parsers, OCR, vision models	Layout, tables, reading order
Semi structured web	HTML pages, wikis, help centres	Crawlers, HTML to text	Boilerplate, navigation noise
Collaboration tools	Confluence, SharePoint, Notion, Slack	Official APIs and webhooks	Permissions, threads, rate limits
Structured data	SQL tables, CRM records, spreadsheets	Row to text templates, or text to SQL	Numbers lose meaning as prose
Code and technical	Repositories, API specs, logs	Syntax aware parsers	Versions and dependencies
Streams	Tickets, chat, email, news feeds	Event consumers	Volume, noise, retention

COMPONENT 1

Connector patterns: pull, push and change data capture



Scheduled pull

A job crawls or queries the source on a schedule and compares against the last snapshot. Simple and universal, but freshness is limited by the interval and full scans are costly.



Event push

The source calls a webhook on create, update and delete. Near real time and efficient, but depends on the source supporting events and on reliable delivery with retries.



Change data capture

Read the database transaction log to stream every change. The gold standard for databases; tools such as Debezium implement it without modifying applications.

COMPONENT 1

Incremental ingestion and change detection



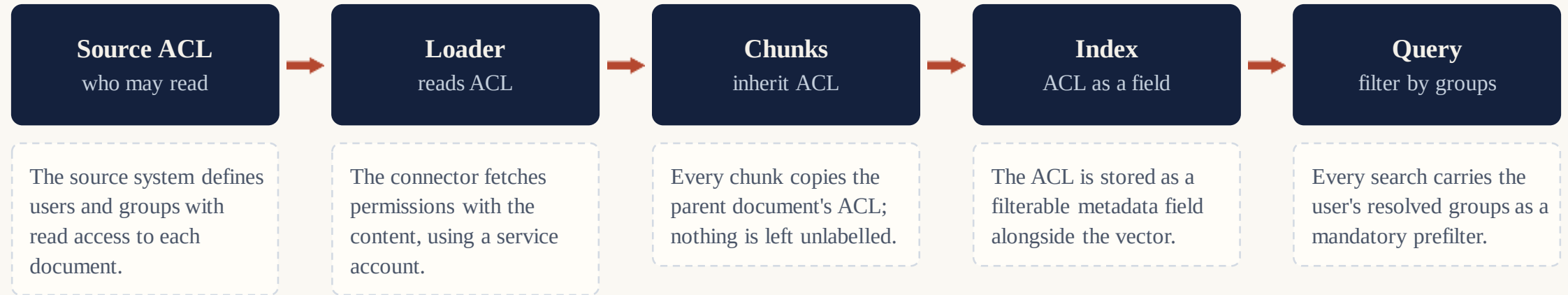
COMPONENT 1

Metadata to capture at the source

Metadata field	Example value	Used for
source_uri	sharepoint://hr/handbook_v7.pdf	Citations and deep links
doc_id and version	handbook, v7	Deduplication and supersession
title and section_path	Contractor Handbook, 7 Leave	Context headers and display
created_at, modified_at	2026 03 14	Recency filters and ranking boosts
author, owner, department	HR Operations	Filters and routing
acl_groups	all_staff, uk_office	Permission filtering at query time
language, doc_type	en, policy	Routing and filters

COMPONENT 1

Inheriting access control through the pipeline



COMPONENT 1

Ingestion checklist

1

Inventory every source

Owner, format, volume, update frequency and access model for each.

3

Define the document identity

A stable ID and version scheme so updates replace rather than duplicate.

5

Implement deletes

Tombstones and propagation to chunks, vectors and caches.

2

Choose a connector pattern

Pull, push or change data capture per source, with retries and dead letter queues.

4

Capture metadata and ACLs

At extraction time, validated against a schema.

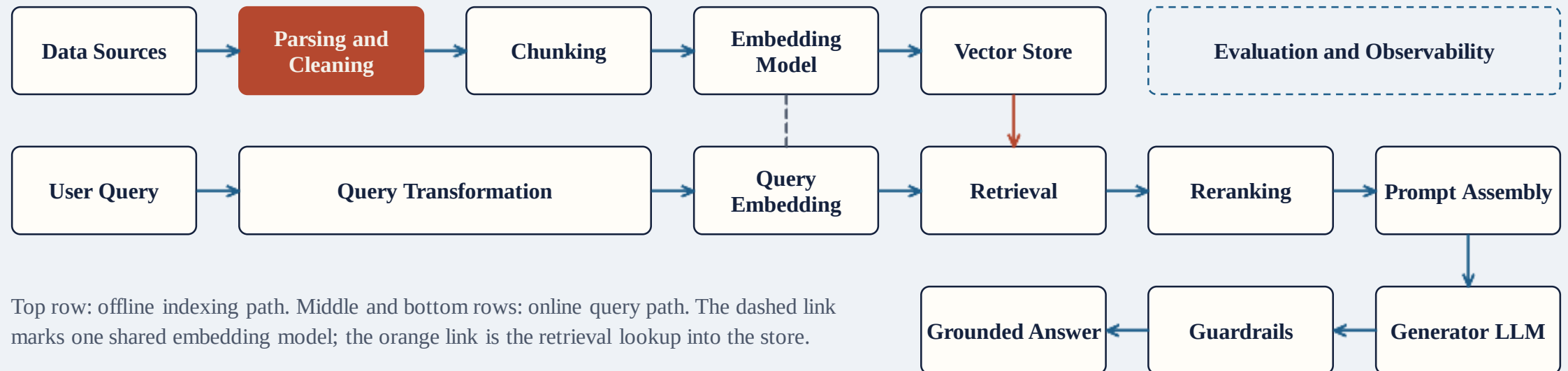
6

Monitor freshness

Index lag, failure counts and coverage per source on a dashboard.

Parsing and Cleaning

Converting heterogeneous files into clean, structured, faithful text: layout analysis, tables, OCR, normalisation, deduplication and enrichment.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

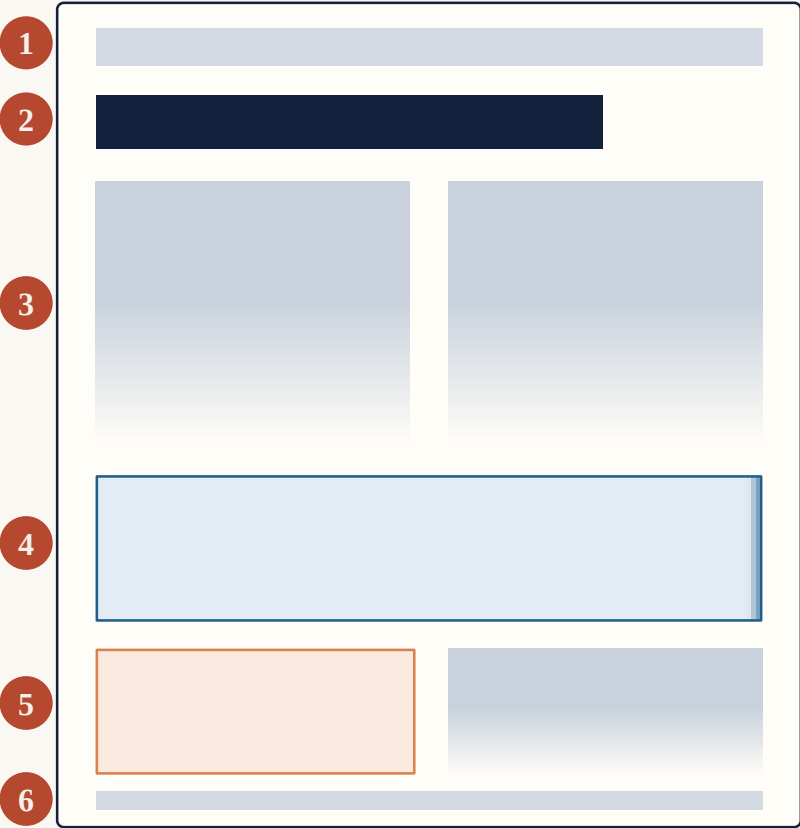
COMPONENT 2

Parsing challenges by format

Format	Typical problem	Recommended approach
Digital PDF	Multi column reading order, headers, footers	Layout aware parsers such as Unstructured, Docling or PyMuPDF with layout
Scanned PDF, images	No text layer at all	OCR such as Tesseract, or a vision language model
HTML	Menus, ads, cookie banners	Main content extraction such as Trafilatura; keep heading tags
Word documents	Styles carry the structure	Map heading styles to a hierarchy; keep lists and tables
Slides	Sparse text, meaning in visuals	Extract text per slide plus speaker notes; caption images
Spreadsheets	Meaning lives in the grid	Keep headers with rows; describe sheets; consider SQL access

COMPONENT 2

Layout aware parsing, annotated



1. Running header

Repeated on every page: detect and remove, keep as metadata.

2. Title and headings

Build the section hierarchy used for chunk context.

3. Two columns

Read left column fully before right; naive extraction interleaves lines.

4. Table

Keep as structured markdown or HTML with headers attached to rows.

5. Figure and caption

Keep the caption; optionally describe the image with a vision model.

6. Footer and page number

Remove from text; keep the page number for citations.

COMPONENT 2

Strategies for tables

Markdown or HTML table

Serialise the table with its header row. LLMs read markdown tables well; best for small and medium tables.

Row as a record

Emit each row as a sentence with header names, such as Plan: Pro, Price: 20. Excellent for lookup queries.

Summary plus raw

Embed an LLM written summary of the table for retrieval; pass the raw table to the generator.

Table as a database

Load large tables into SQL and give the agent a query tool instead of embedding them.

Keep captions and context

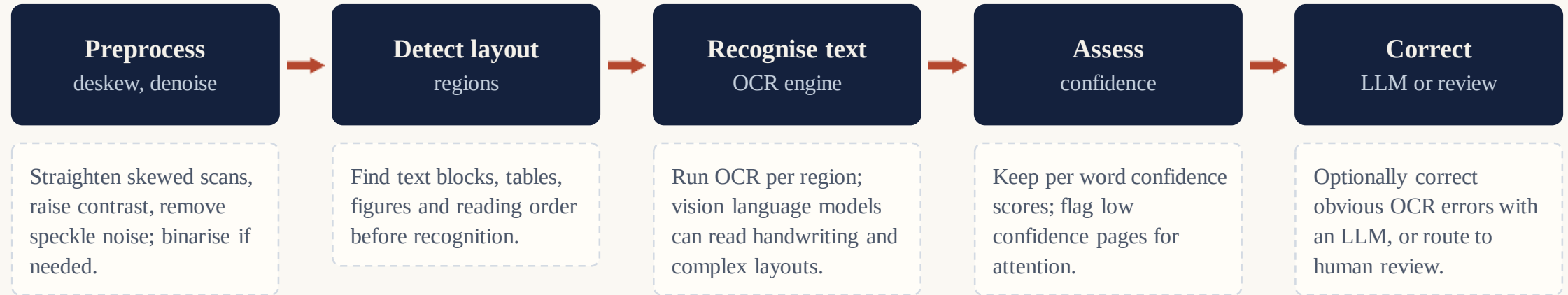
Attach the table title, caption and surrounding paragraph to every table chunk.

Never split mid table

If a table must be split, repeat the header row in every piece.

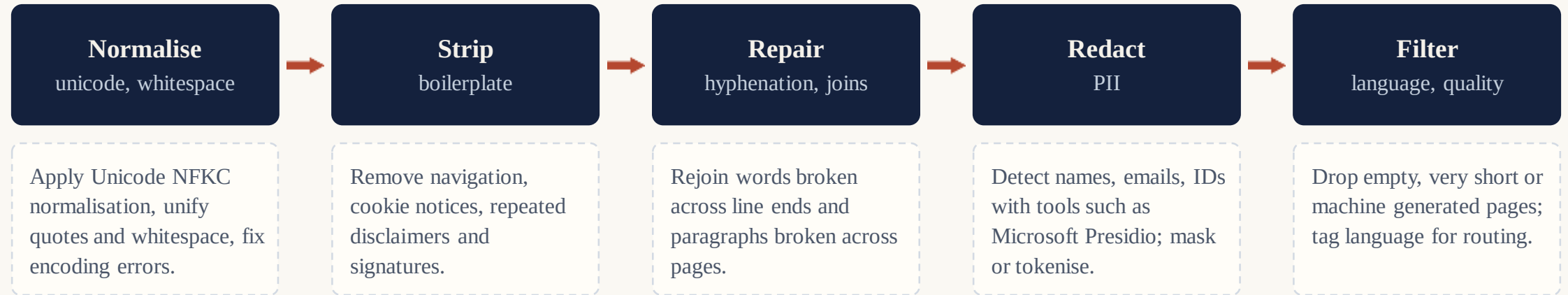
COMPONENT 2

OCR pipeline for scanned documents



COMPONENT 2

Cleaning and normalisation pipeline



Deduplication: exact and near duplicate

Exact deduplication

- Hash the normalised text of each document or chunk.
- Identical hashes are duplicates; keep one canonical copy.
- Fast and precise, but misses near copies with small edits.
- Ideal first pass across mirrored folders and email attachments.

Near duplicate detection

- MinHash with locality sensitive hashing estimates Jaccard similarity of shingles.
- SimHash produces fingerprints where similar texts differ in few bits.
- Embedding similarity above a threshold catches paraphrases.
- Resolve clusters by keeping the latest or most authoritative version.

COMPONENT 2

Enrichment at indexing time



Titles and headings

Recover or generate a descriptive title for every document and section.



Summaries

A short LLM summary per document or section, used for routing and hierarchical retrieval.



Hypothetical questions

Generate questions each chunk answers and index them, which narrows the query document gap.



Entities and keywords

Extract people, products, codes and dates for filters and keyword boosts.



Classification

Tag document type, topic and audience for routing and filtering.

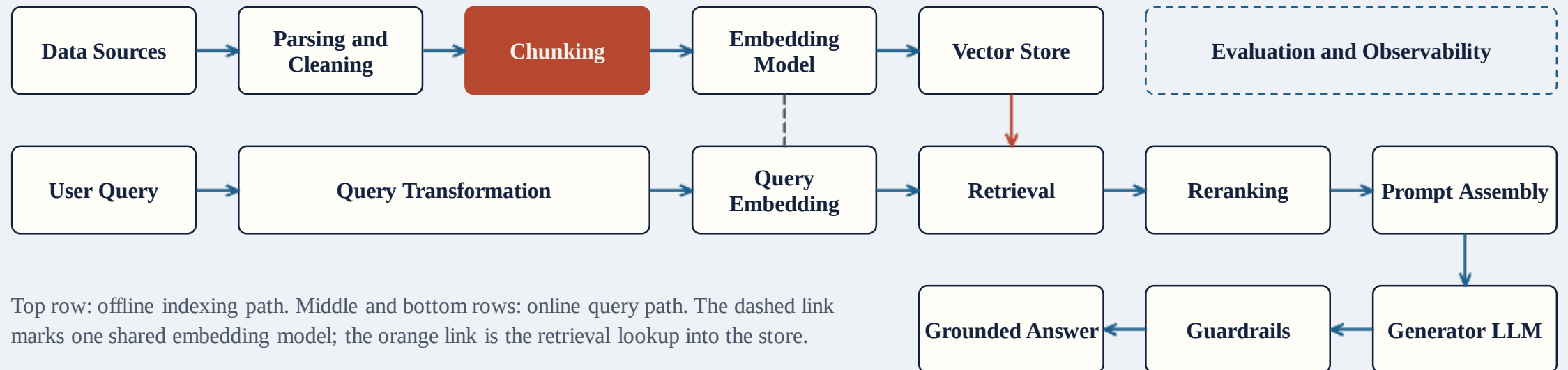


Quality signals

Authority, freshness and popularity scores used as ranking features.

Chunking

Splitting documents into units that are small enough to be precise and large enough to be meaningful: the most underestimated design decision in RAG.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

COMPONENT 3

The granularity trade off

Sentence

Paragraph

Section

Page

Document

Smaller chunks

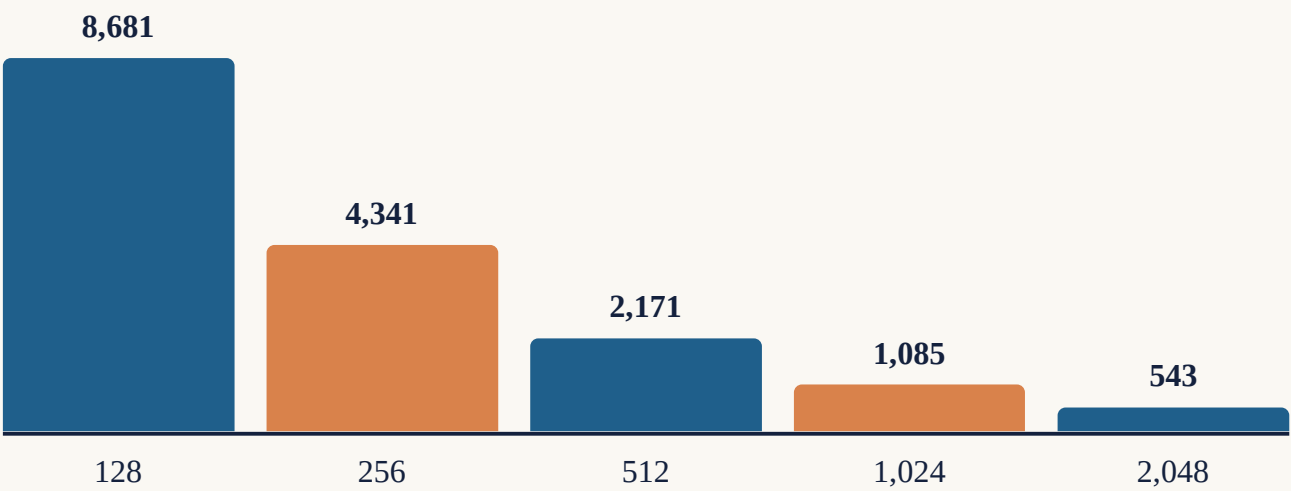
- Sharper embeddings focused on one idea.
- Higher precision; less noise in the prompt.
- Risk: missing context, split answers, orphaned references.
- More vectors to store and search.

Larger chunks

- More surrounding context in each unit.
- Better for questions needing explanation or synthesis.
- Risk: diluted embeddings that match nothing sharply.
- Fewer slots fit in the context budget.

COMPONENT 3

Chunk size arithmetic



Number of chunks for a one million token corpus by chunk size in tokens, with 10 percent overlap.
Computed values.

The arithmetic

Chunks = 1 + (corpus minus size) divided by stride, rounded up, with stride = $0.9 \times \text{size}$ for 10 percent overlap.

Storage

At 768 float32 dimensions each vector is 3,072 bytes: 8,681 chunks need 26.7 MB, 543 need 1.7 MB.

Context

With $k = 5$, the generator reads 640 tokens at size 128 but 10,240 tokens at size 2,048.

COMPONENT 3

Fixed size chunking

Chunk 1 | 512 tokens

Chunk 2 | 512 tokens

Chunk 3 | 512 tokens

Chunk 4 | rest

WHAT THE BOUNDARY CAN DO TO A SENTENCE

...end of chunk 2: “Employees on probation are entitled to a notice period of”

...start of chunk 3: “one week, rising to three months after the first year.”

Neither chunk alone answers “what is the notice period during probation?”

Strengths

Trivial to implement, predictable sizes, fast, a solid baseline for any corpus.



Weaknesses

Ignores meaning and structure; cuts sentences, lists and tables at arbitrary points.



COMPONENT 3

Chunk overlap

Without overlap: content at a boundary belongs wholly to one side.



With overlap: the tail of each chunk is repeated at the head of the next.





Why overlap

Sentences near boundaries appear in two chunks, so at least one contains them with context.



How much

Typically 10 to 20 percent of chunk size; 50 to 100 tokens for 512 token chunks.



The cost

Storage and duplicates grow with overlap; retrieved neighbours may repeat text.

COMPONENT 3

Recursive splitting: separators in priority order



COMPONENT 3

Structure aware chunking

Markdown and HTML

Split on heading levels; each chunk keeps its heading path, such as Guide, Install, Linux.

Code

Split on functions and classes using a syntax tree parser such as tree sitter, never mid function.

Legal and regulatory

Split on articles, clauses and subclauses; keep numbering and definitions linked.

Conversations

Split on turns or threads; keep speaker names and timestamps with the text.

Slides

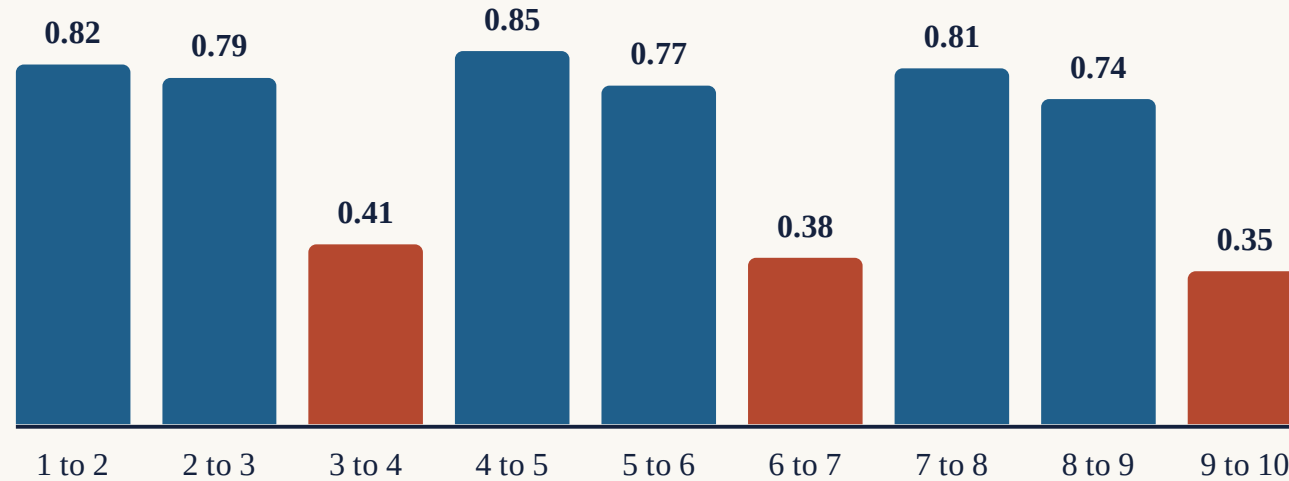
One slide per chunk, including title and speaker notes.

FAQs

One question and answer pair per chunk: the ideal retrieval unit.

COMPONENT 3

Semantic chunking



Similarity between adjacent sentences in a toy ten sentence passage. Illustrative values chosen for teaching.

Method

Embed each sentence, compute cosine similarity between neighbours, and cut where similarity drops below a threshold.

Reading the chart

Orange bars fall below 0.5: topic shifts after sentences 3, 6 and 9, giving three coherent chunks.

Trade off

Better topical coherence, but one embedding call per sentence and sizes that vary widely.

COMPONENT 3

Small to big: parent and child chunks

Parent chunk: 2,000 tokens, the full section

Stored in a document store, returned to the LLM when any child matches



Child 1

400 tokens, embedded and indexed for search

Child 2

400 tokens; a query matches this child precisely

Child 3

400 tokens, embedded and indexed for search

Child 4

400 tokens, embedded and indexed for search

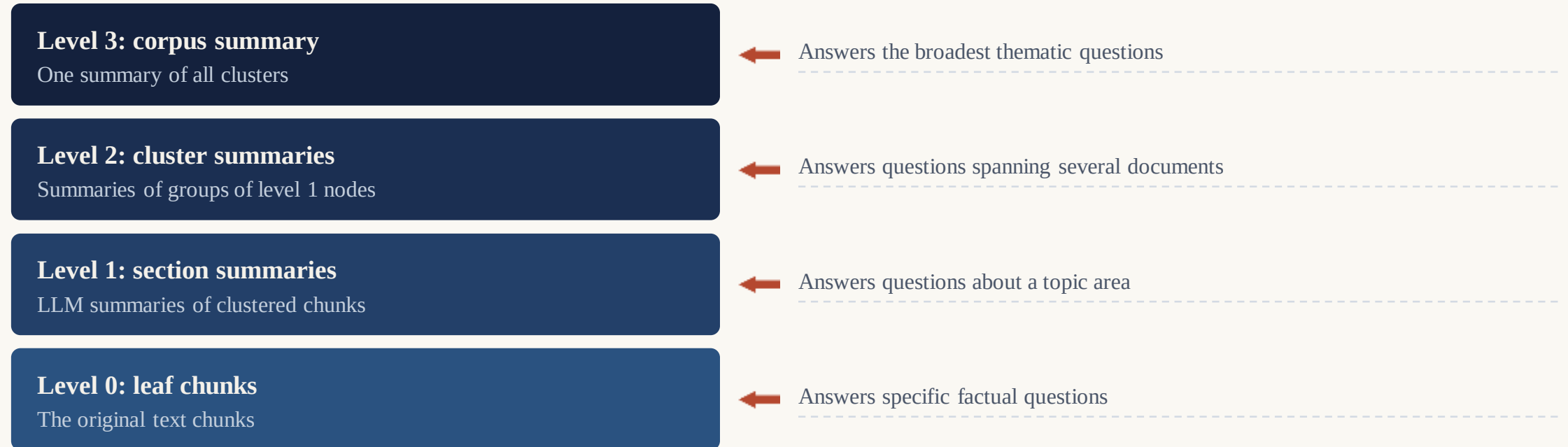
Child 5

400 tokens, embedded and indexed for search

Sentence window variant: index single sentences, and on a match return the sentence together with the k sentences either side.

COMPONENT 3

Hierarchical chunking and summary trees



RAPTOR (Sarathi et al., 2024) builds this tree by recursively embedding, clustering and summarising, then retrieves across all levels at once.

COMPONENT 3

Giving chunks their missing context

Contextual chunk headers

- Prepend document title and section path to each chunk before embedding.
- Anthropic's contextual retrieval (2024) goes further: an LLM writes a short sentence situating each chunk within its document.
- The context is embedded and BM25 indexed with the chunk.
- Anthropic reported substantially fewer retrieval failures, especially combined with reranking.

Late chunking

- Run a long context embedding model over the whole document first.
- Then pool token embeddings per chunk span, after attention has seen everything.
- Each chunk vector now carries information from the rest of the document.
- Proposed by Jina AI (Günther et al., 2024); needs a long context embedder.

COMPONENT 3

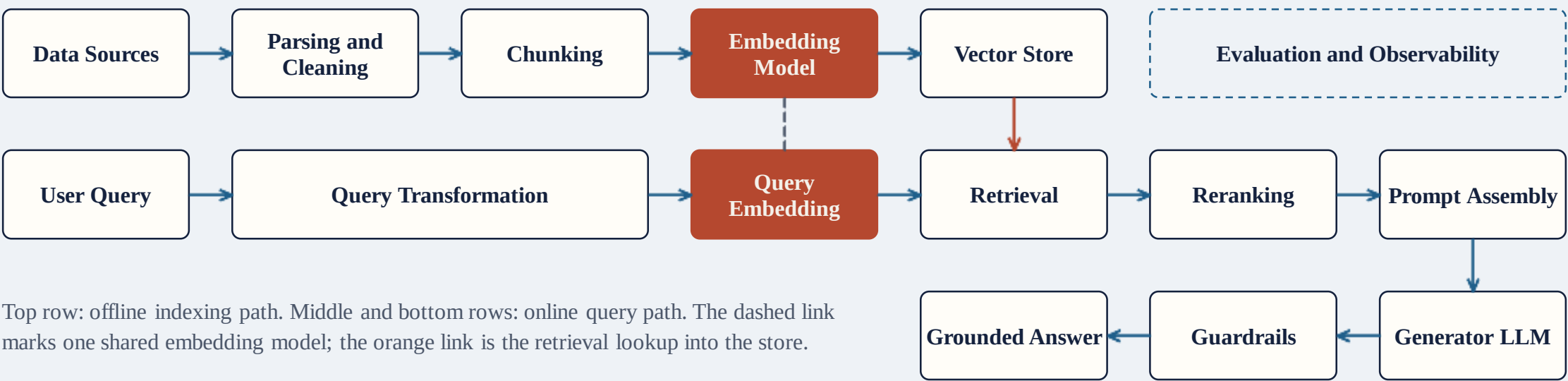
Comparing chunking strategies

Strategy	Best for	Cost	Main risk
Fixed size	Baselines, homogeneous prose	Very low	Arbitrary cuts
Recursive	General purpose default	Very low	Ignores document semantics
Structure aware	Markdown, HTML, code, legal	Low	Needs good parsing
Semantic	Long unstructured prose	Medium	Unstable sizes
Small to big	Precise search, rich answers	Low	Larger prompts
Hierarchical	Broad and thematic questions	High	Summary errors
Contextual headers	Ambiguous, reference heavy text	Medium	Indexing LLM cost

Selection rule: start with structure aware or recursive splitting at 256 to 512 tokens, add contextual headers, then test small to big. Let your golden dataset decide.

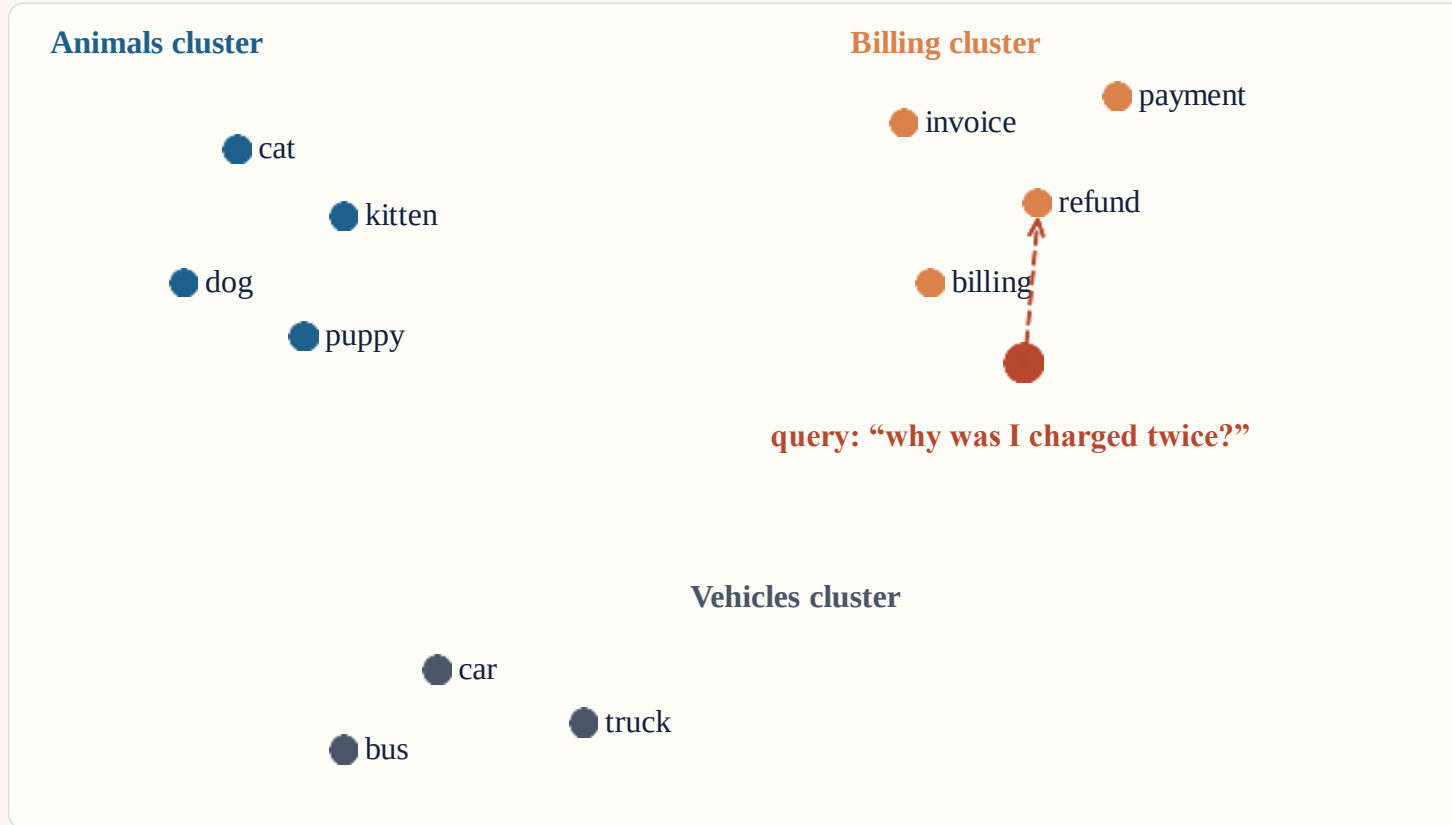
Embedding Models

The model that turns text into vectors so that similar meanings land close together. The same model must embed both chunks and queries.



COMPONENT 4

What an embedding is



Geometry is meaning

Texts with similar meaning map to nearby points; distance approximates semantic dissimilarity.

No shared words needed

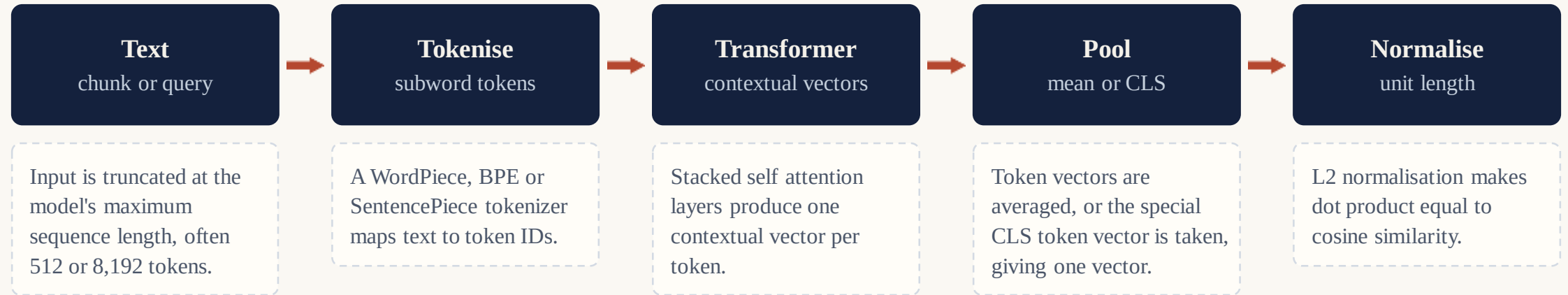
The query shares no words with "refund", yet lands beside it.

A 2D sketch

Real embeddings have hundreds or thousands of dimensions; this projection is schematic.

COMPONENT 4

From text to vector: the encoder pipeline



COMPONENT 4

Bi encoders, cross encoders and late interaction

Property	Bi encoder	Cross encoder	Late interaction (ColBERT)
Input	Query and document encoded separately	Query and document encoded together	Separately, one vector per token
Output	One vector each; compare by cosine	One relevance score per pair	MaxSim over token vectors
Precompute documents?	Yes	No	Yes
Speed at query time	Very fast, millions of documents	Slow, tens to hundreds of pairs	Fast, with larger storage
Accuracy	Good	Best	Very good
Role in RAG	First stage retrieval	Reranking	Retrieval or reranking

COMPONENT 4

How embedding models learn: contrastive training

Embedding models are trained with contrastive learning. Each training example has an anchor, such as a question, a positive, such as the passage that answers it, and many negatives.

The InfoNCE loss rewards high similarity between anchor and positive relative to all negatives in the batch, which is why large batch sizes help.

Hard negatives, passages that look relevant but are not, teach fine distinctions and are mined with BM25 or an earlier model.

Fine tuning an embedder on your own question and passage pairs is one of the highest leverage improvements for specialised domains.

THE OBJECTIVE, IN WORDS

loss = negative log of

$\exp(\text{sim}(\text{anchor}, \text{positive}) / \tau)$

divided by

Σ over all candidates of $\exp(\text{sim}(\text{anchor}, \text{candidate}) / \tau)$

τ is a temperature that sharpens the distribution

Other batch items act as free negatives

Hard negatives come from BM25 or a prior model

Result: positives pulled close, negatives pushed away

Similarity measures

$$\cos(q, d) = (q \cdot d) / (\|q\| \|d\|)$$

For unit normalised vectors, all three measures produce the same ranking

Cosine similarity

Angle between vectors; ranges from minus one to one. Ignores magnitude. The default for text.

Dot product

$q \cdot d$, the sum of elementwise products. Equals cosine when both vectors are unit length.

Euclidean distance

Straight line distance. For unit vectors, squared distance equals $2 - 2 \cos$, so rankings match.

COMPONENT 4

Worked example: cosine similarity in three dimensions

Vector	Components	$q \cdot d$	$\ d\ $	Cosine with q	Rank
Query q	(1, 2, 2)		3.000		
Document d1	(2, 3, 1)	10	3.742	$10 / 11.225 = 0.891$	2
Document d2	(3, 0, 1)	5	3.162	$5 / 9.487 = 0.527$	3
Document d3	(0, 2, 2)	8	2.828	$8 / 8.485 = 0.943$	1

0.943

d3 is the nearest neighbour of q

1.000

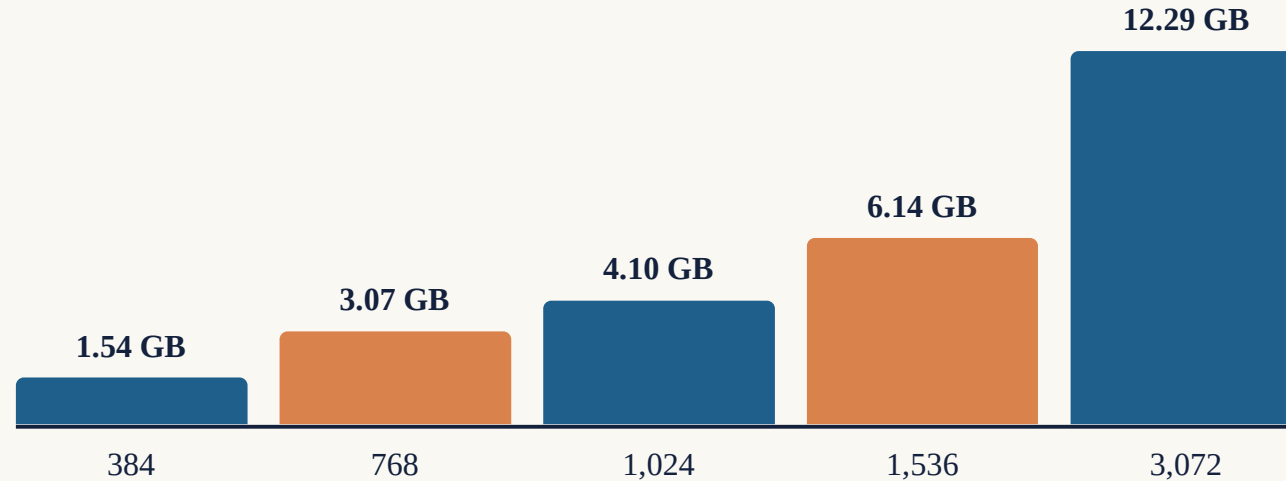
Euclidean distance from q to d3, also the smallest

3

dimensions here; real models use 384 to 3,072

COMPONENT 4

Dimensionality, memory and Matryoshka embeddings



Raw float32 memory for one million vectors by embedding dimension. Computed values, excluding index overhead.

The arithmetic

Memory = vectors $\times$ dimensions $\times$ 4 bytes for float32. One million $\times$ 768 $\times$ 4 = 3.07 GB.

Matryoshka embeddings

Models trained with Matryoshka representation learning keep most quality when truncated to a prefix of the dimensions.

Practical use

Search with short truncated vectors, then rescore the top candidates with the full vectors.

COMPONENT 4

Choosing an embedding model



Retrieval quality on your data

Measure recall at k on your golden set; leaderboards are only a shortlist.



Domain and language

Legal, biomedical, code and multilingual corpora may need specialised models.



Maximum input length

Must exceed your chunk size in that model's tokens, or text is truncated.



Dimension and cost

Drives storage, memory and latency at your corpus size.



Hosting and licence

API service versus self hosted open weights; data residency and terms of use.



Throughput and latency

Indexing speed for large corpora; query embedding latency on the critical path.

COMPONENT 4

Benchmarks: MTEB and its limits

What MTEB offers

- The Massive Text Embedding Benchmark, Muennighoff et al., 2022, spans many tasks and datasets.
- Retrieval, reranking, clustering, classification and similarity are scored separately.
- Look at the retrieval columns specifically, not the overall average.
- Multilingual and domain specific variants exist.

Why you still need your own evaluation

- Public datasets may have leaked into training data of some models.
- Benchmark queries rarely resemble your users' questions.
- Your documents have their own vocabulary, formats and lengths.
- Rankings between close models often flip on private data.

COMPONENT 4

Sparse, dense and learned sparse representations

Representation	How it works	Strength	Weakness
Sparse lexical (BM25)	Vocabulary sized vector of weighted term counts	Exact terms, codes, names; no training	No synonyms or paraphrase
Dense	A few hundred to a few thousand learned dimensions	Meaning, paraphrase, cross lingual	Weak on rare exact tokens
Learned sparse (SPLADE)	A model predicts weights over the vocabulary, including expansion terms	Semantic and interpretable; inverted index	Needs a trained model
Multi vector (ColBERT)	One vector per token, MaxSim scoring	Fine grained matching	Large storage footprint

COMPONENT 4

Embedding operations in production



Version everything

Store the model name and version with every vector. Never mix vectors from two models in one index.



Plan reembedding

Changing the model means reembedding the whole corpus. Build a new index in parallel, evaluate, then switch atomically.

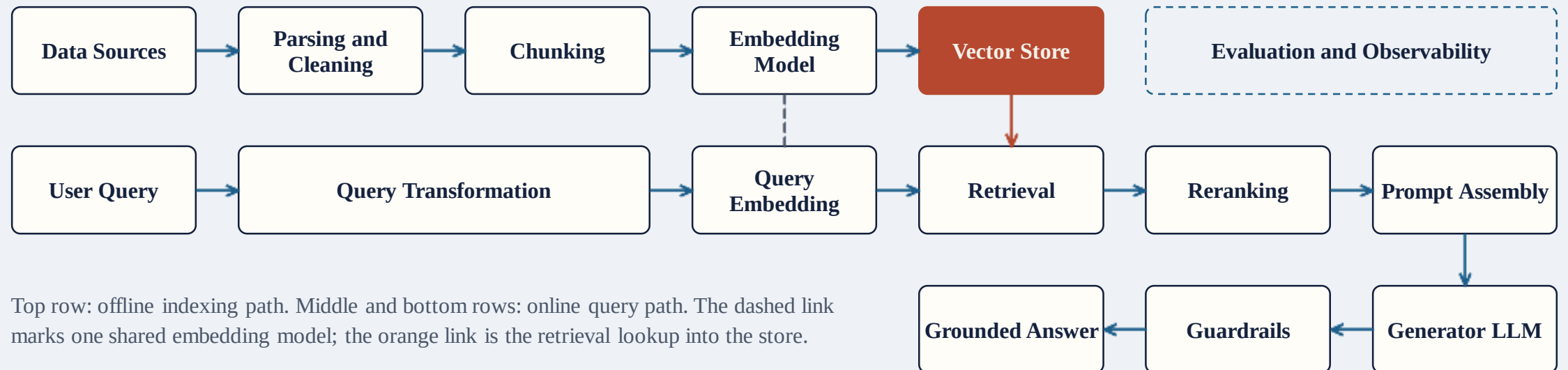


Cache embeddings

Key by content hash and model version, so unchanged chunks are never embedded twice.

Vector Store and Index

The store that holds vectors, text and metadata and answers nearest neighbour queries in milliseconds across millions or billions of items.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

COMPONENT 5

What a vector store does, layer by layer

API and query layer

Search, filter, upsert, delete, hybrid queries



What your application calls: REST, gRPC or SQL

Filtering engine

Metadata predicates combined with vector search



Where access control and recency filters run

ANN index

HNSW, IVF, product quantisation, DiskANN



Where the speed comes from; trades recall for latency

Storage

Vectors, payloads, write ahead log, snapshots



Durability, persistence and recovery

Distribution

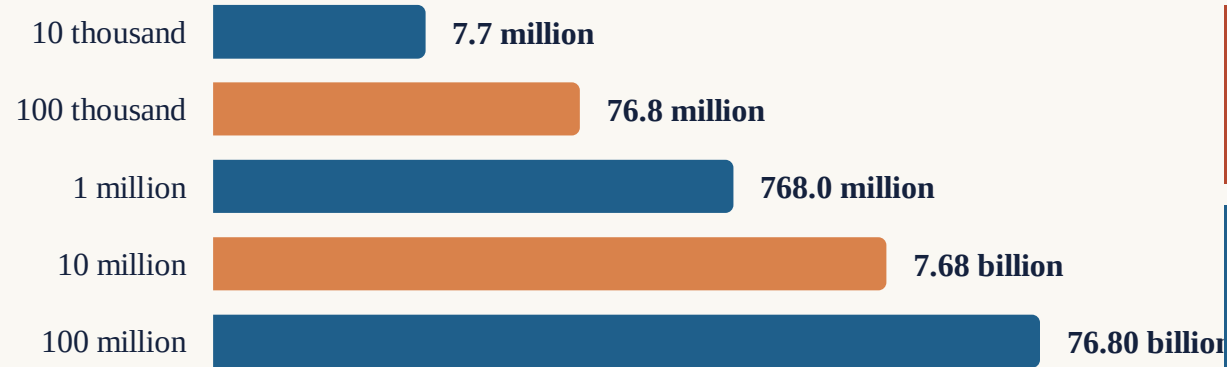
Sharding, replication, tenancy



How the system scales beyond one machine

COMPONENT 5

The cost of exact nearest neighbour search



Multiply add operations per exact query at 768 dimensions, by corpus size. Computed; bar length on a logarithmic scale.

The arithmetic

Exact search compares the query with every vector: $n \times d$ multiply add operations per query.

Why it breaks

At 100 million vectors that is 76.8 billion operations per query, before any concurrency.

The escape

Approximate nearest neighbour indexes visit a tiny fraction of vectors and accept a small recall loss.

COMPONENT 5

The approximate nearest neighbour landscape



Graph based

HNSW, DiskANN, NSG. Navigate a proximity graph greedily toward the query. The current default for high recall at low latency.



Clustering based

IVF. Partition vectors into cells around centroids and search only the nearest cells.



Quantisation

PQ, scalar and binary quantisation. Compress vectors to fit memory and speed up distance computation.



Tree based

KD trees, Annoy's random projection forests. Effective at low dimension, weaker at high dimension.



Hashing

Locality sensitive hashing places similar vectors in the same buckets. Simple guarantees, usually lower recall.

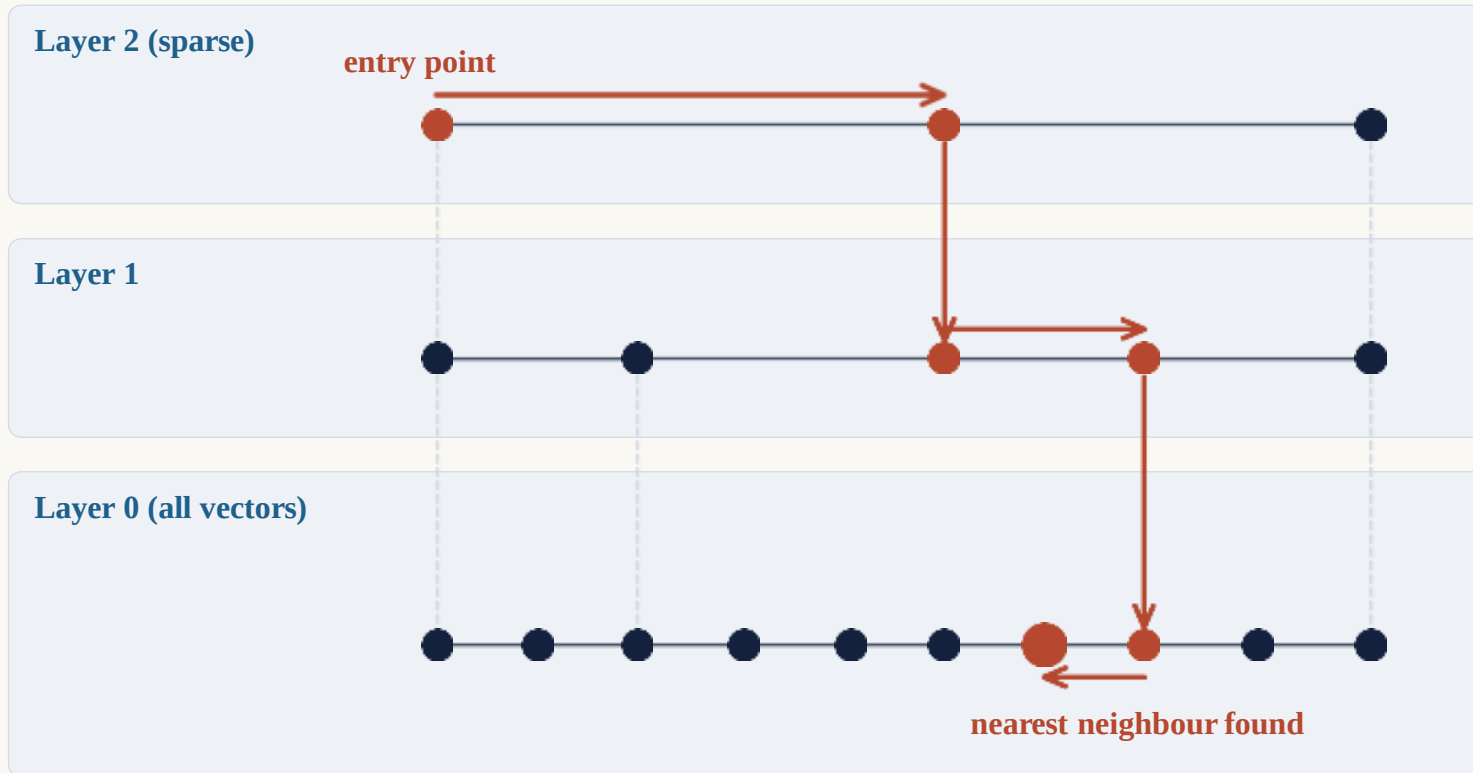


Hybrids

IVF with PQ, HNSW with quantised vectors, disk graphs with compressed codes. Most production indexes combine families.

COMPONENT 5

HNSW: navigating a layered graph



Hierarchy

Each vector is inserted into layer 0 and, with exponentially decaying probability, into higher layers.

Greedy descent

Search starts at the top entry point, moves to the closest neighbour, then drops a layer and repeats.

Why it is fast

Upper layers make long jumps, lower layers refine locally: roughly logarithmic search cost.

COMPONENT 5

HNSW parameters and how to tune them

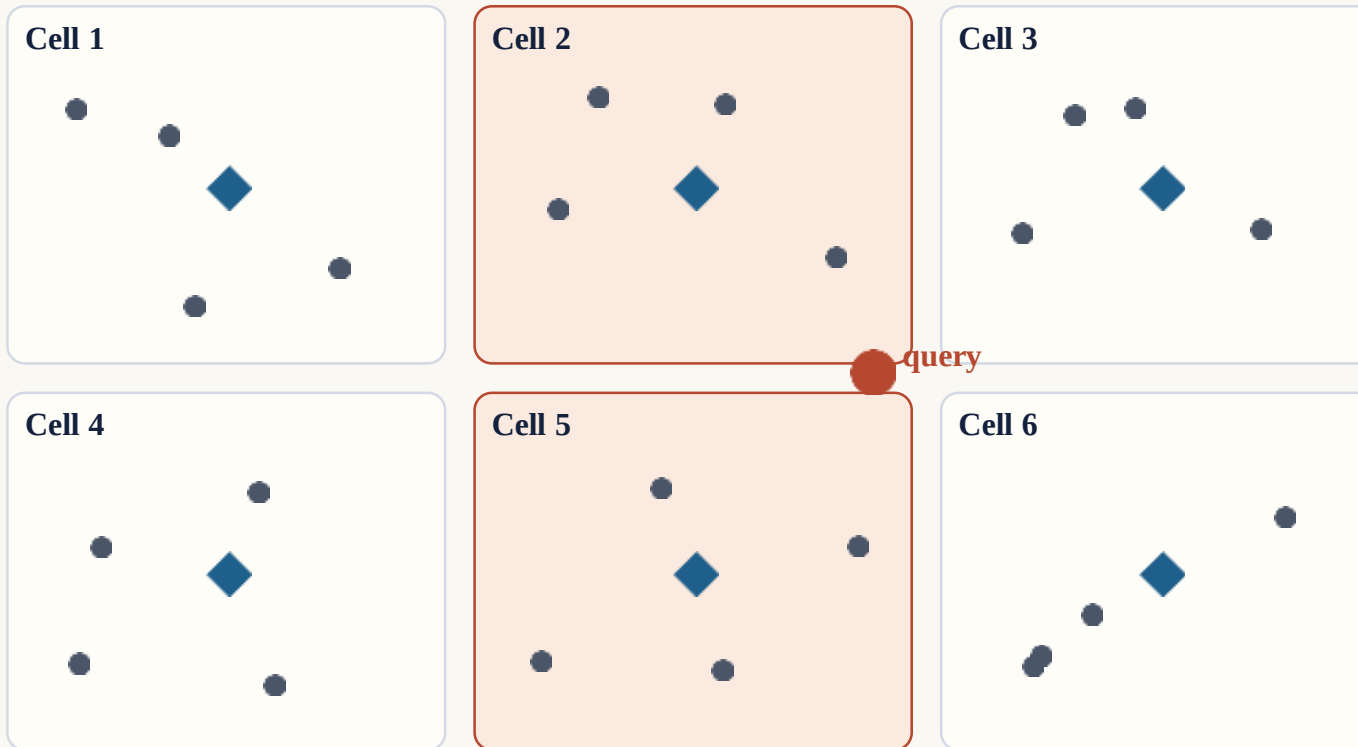
Parameter	Meaning	Increase it to	Cost of increasing it
M	Maximum links per node per layer, often 16 to 64	Raise recall, especially at high dimension	More memory and slower builds
efConstruction	Candidate list size while inserting, often 100 to 500	Build a higher quality graph	Slower indexing only
efSearch	Candidate list size while querying; must be at least k	Raise recall per query	Higher query latency
Distance metric	Cosine, dot product or L2	Match the embedding model's training	None, but mismatch hurts recall

Tuning procedure

Fix M and efConstruction at build time, then sweep efSearch at query time while measuring recall at k against exact search, and pick the smallest value meeting your recall target.

COMPONENT 5

IVF: searching only the nearest clusters



Build

k means clusters the vectors into nlist cells; each vector is stored in the inverted list of its nearest centroid (diamonds).

Search

Compare the query with all centroids, then scan only the nprobe nearest cells, here the two shaded cells.

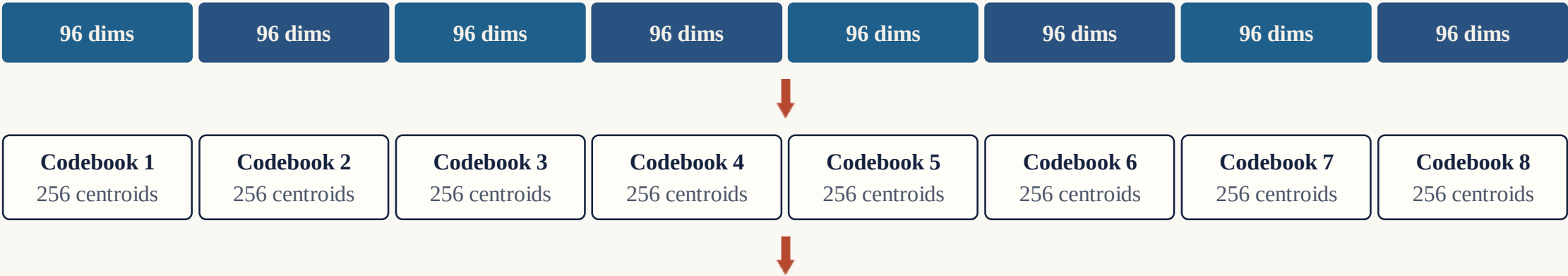
Trade off

Higher nprobe raises recall and latency. A query near a cell border needs nprobe above 1.

COMPONENT 5

Product quantisation, step by step

1. Split a 768 dimensional float32 vector (3,072 bytes) into 8 subvectors

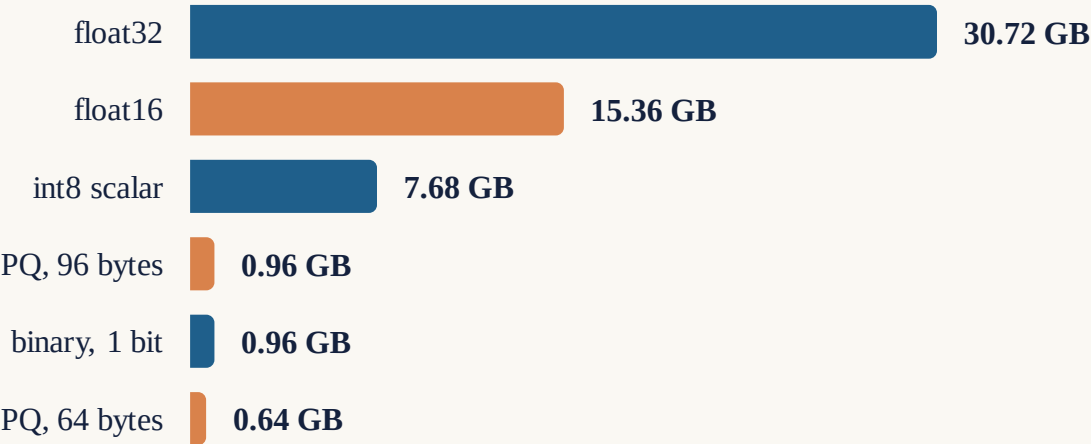


2. Replace each subvector with the ID of its nearest centroid: 8 bytes in total, a 384 fold compression



COMPONENT 5

Memory footprint by vector representation



Raw memory for ten million 768 dimensional vectors by representation. Computed values.

The arithmetic

Ten million vectors × 768 dimensions: 4 bytes each is 30.72 GB; 1 byte is 7.68 GB; 1 bit is 0.96 GB.

Graph overhead

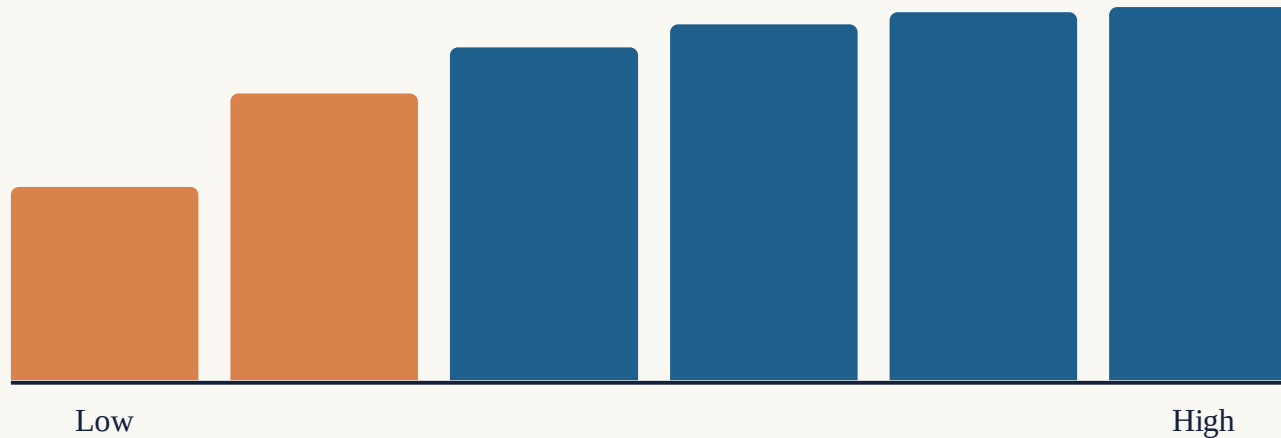
HNSW adds link storage: with M equal to 16, roughly 32 links × 4 bytes per vector at layer 0, about 1.28 GB more.

Recover accuracy

Search compressed vectors, then rescore the top few hundred with full precision vectors.

COMPONENT 5

The recall versus latency trade off



Schematic: recall at k (bar height) as the search effort parameter increases from low to high. Shape only, no measured values.

The shape

Recall rises steeply at first, then flattens: each extra point of recall costs more latency than the last.

The knob

efSearch in HNSW, nprobe in IVF, rescoring depth with quantisation.

Operating point

Choose the knee of the curve that meets your recall target within the latency budget.

COMPONENT 5

Combining vector search with metadata filters

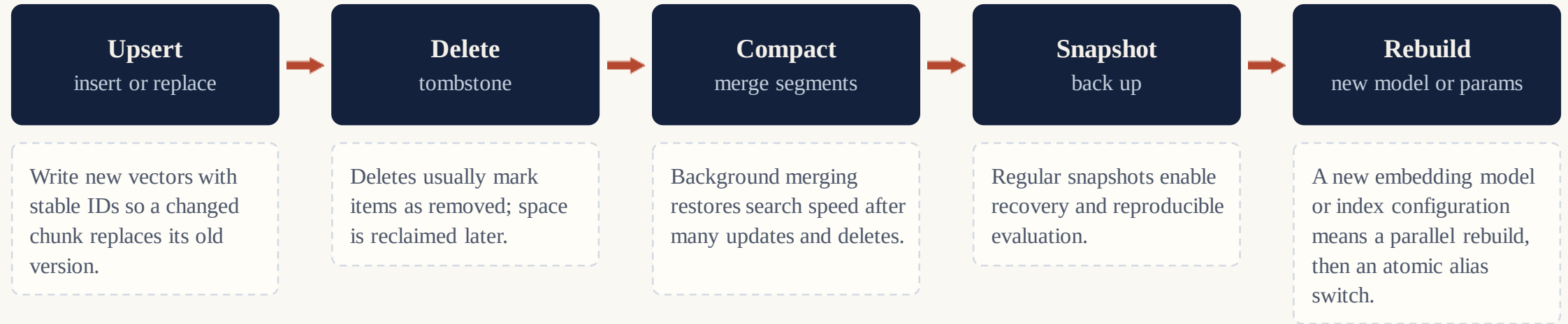
Strategy	How it works	Risk	When to use
Post filtering	Search top k, then discard results failing the filter	May return far fewer than k results for selective filters	Filters that remove few items
Pre filtering	Apply the filter first, then search only the allowed subset	Brute force on large subsets is slow; graph connectivity breaks	Very selective filters, small subsets
Filtered or in index search	The ANN traversal checks the filter as it explores	Engine specific; needs filter aware indexes	The default in modern vector databases
Partitioning	Separate indexes or namespaces per tenant or category	Many small indexes to manage	Hard boundaries such as tenants

Vector store options compared

Option	Type	Notable characteristics
FAISS	Library	Many index types, GPU support; no database features such as filtering or persistence built in
pgvector	PostgreSQL extension	Vectors beside relational data, SQL filters and joins, transactions; HNSW and IVFFlat
Qdrant	Open source database	Rust, rich payload filtering, quantisation, hybrid sparse and dense
Weaviate	Open source database	Built in hybrid search, modules for vectorisation, multitenancy
Milvus	Open source database	Distributed architecture for very large scale, many index types, GPU indexes
Elasticsearch, OpenSearch	Search engines	Mature BM25 plus vector fields; strong for hybrid search
Pinecone	Managed service	Fully managed, serverless option, no infrastructure to operate
Chroma	Open source, embedded	Simple developer experience for prototypes and small applications

COMPONENT 5

The index lifecycle



COMPONENT 5

Sharding, replication and multitenancy



Sharding

Split the collection across nodes by hash, tenant or time. Queries fan out to shards and results are merged, so latency follows the slowest shard.



Replication

Copies of each shard serve reads in parallel and survive node failures. Throughput scales with replicas.

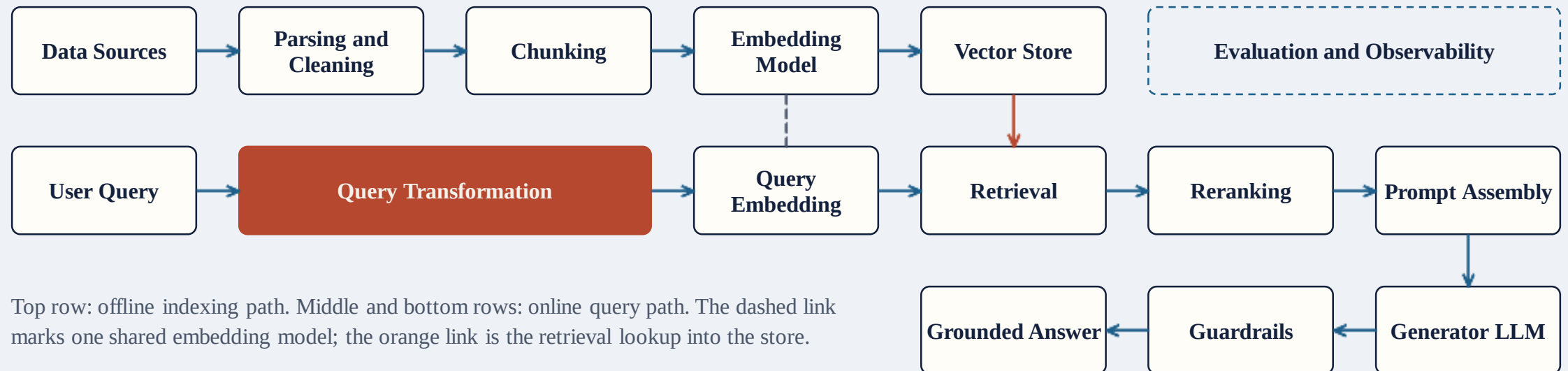


Multitenancy

Isolate customers with a tenant field filter, a namespace per tenant, or a collection per tenant, trading isolation against overhead.

Query Transformation

Turning what the user typed into what the retriever needs: rewriting, expansion, multiple queries, hypothetical documents, decomposition and routing.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

COMPONENT 6

Why raw queries retrieve poorly



Conversational references

“What about for contractors?” is meaningless without the previous turn.



Vocabulary mismatch

Users say “charged twice”; documents say “duplicate billing”.



Multiple intents

“Compare the 2025 and 2026 leave policies” needs two retrievals.



Implicit filters

“UK policy updated this year” hides a region and a date filter.



Noise and typos

Greetings, pleasantries and misspellings dilute the query embedding.



Wrong abstraction level

A very specific question may be better served by first retrieving general principles.

COMPONENT 6

Query rewriting and expansion

REWRITING AND EXPANSION, BY EXAMPLE

Chat history: “Tell me about our parental leave policy.”

User: “what about contractors??”

→ Condensed: “parental leave policy for contractors”

→ Expanded: “parental leave, maternity, paternity, family leave, contractors, non employees, freelancers”

→ Keyword form for BM25: “parental leave contractors”

Condensation

An LLM rewrites a follow up into a standalone question using the chat history.

Expansion

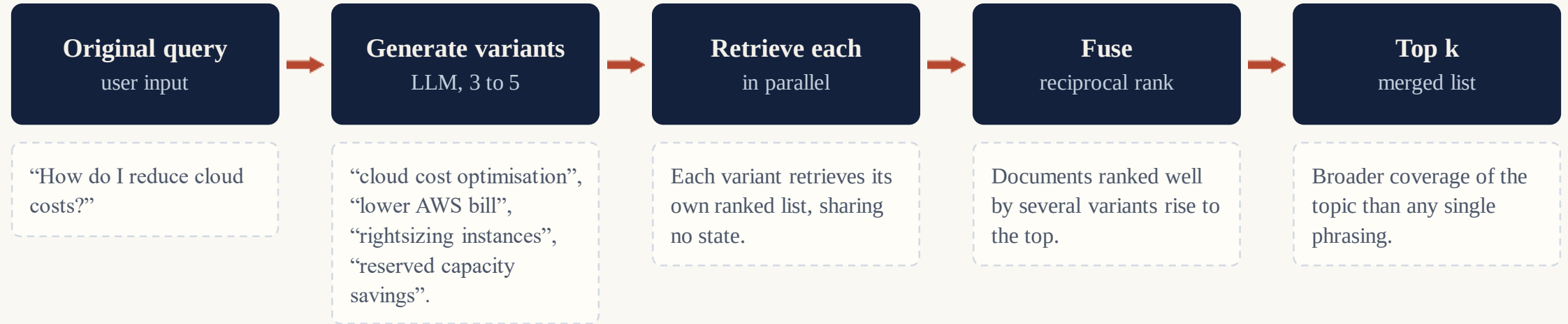
Add synonyms and related terms, from an LLM or a thesaurus, to improve lexical recall.

Risk

Expansion can drift the query off topic; weight the original query highest when fusing results.

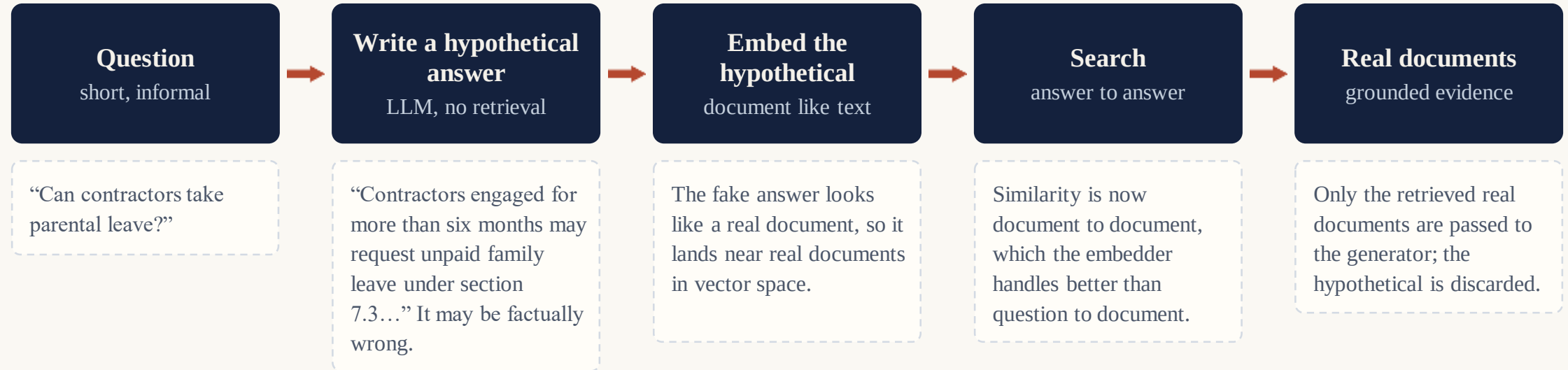
COMPONENT 6

Multi query retrieval and RAG Fusion



COMPONENT 6

HyDE: hypothetical document embeddings



Decomposition and step back prompting

Query decomposition

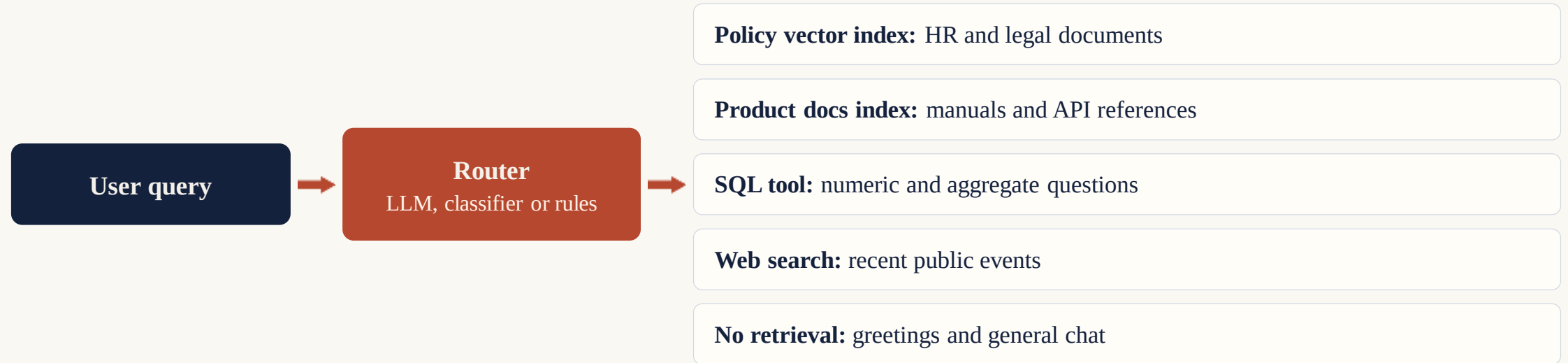
- Break a complex question into sub questions answered separately.
- “Which of our two largest customers renewed later?” becomes: who are the two largest customers; when did each renew.
- Sub answers are combined, sequentially when one depends on another.
- Essential for multihop and comparison questions.

Step back prompting

- Ask a more general question first, then the specific one.
- “Can I claim a laptop bought for home working?” steps back to “What is the equipment expenses policy?”
- Retrieve for both and pass both contexts to the generator.
- Introduced by Zheng et al., 2023, at Google DeepMind.

COMPONENT 6

Query routing



Why route

Different questions need different sources and strategies. Routing avoids searching everything for everything, which adds noise and cost; misrouting is logged and evaluated like any other classifier error.

Query understanding and filter extraction

SELF QUERYING: EXTRACTING STRUCTURED FILTERS

Query: “Show UK policies about remote work updated since January 2026”

→ Semantic part: “remote work policy”

→ Filters: region equals UK

→ Filters: doc_type equals policy

→ Filters: modified_at on or after 1 January 2026

Vector search runs on the semantic part, restricted by the filters.

Mechanism

An LLM is given the metadata schema and returns the semantic query plus a structured filter.

Validation

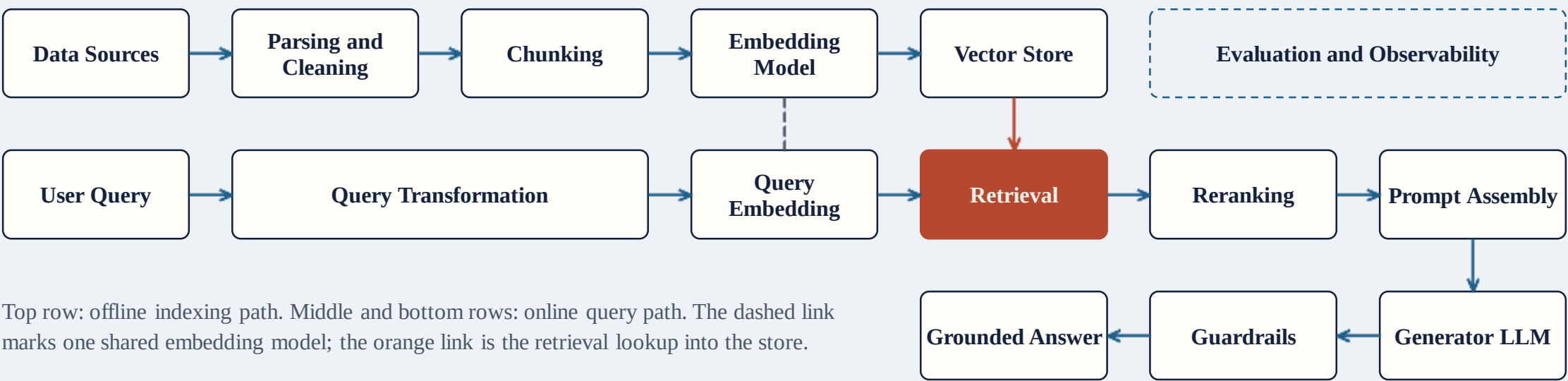
Check field names and values against the schema; drop invalid filters rather than failing.

Benefit

Precise constraints that embeddings handle poorly, such as dates, numbers and categories.

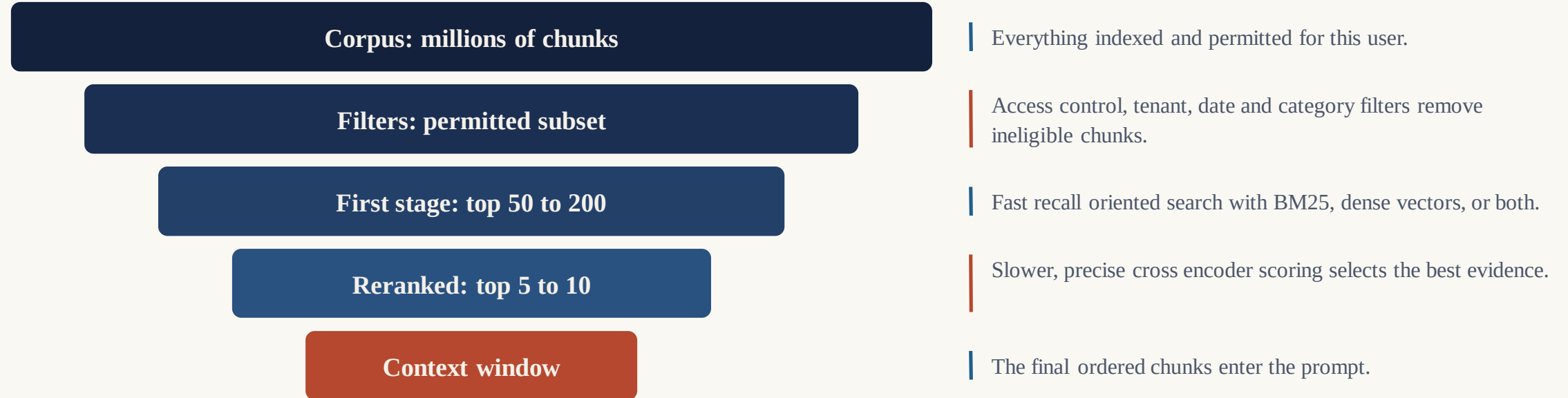
Retrieval

Finding the candidate evidence: lexical, dense and hybrid search, fusion of ranked lists, and how many results to take.



COMPONENT 7

Retrieval as a ranking funnel



Lexical retrieval: BM25

$$\text{score}(D, Q) = \sum \text{IDF}(t) \cdot f(t, D) \cdot (k1 + 1) / (f(t, D) + k1 \cdot (1 \text{ minus } b + b \cdot |D| / \text{avgdl}))$$

Summed over every query term t that appears in document D

$f(t, D)$

Frequency of term t in D ; it saturates, so repeats add less and less.

$\text{IDF}(t)$

$\ln(1 + (N \text{ minus } n + 0.5) / (n + 0.5))$; n of N documents contain t .

$k1$

Controls term frequency saturation, typically 1.2 to 2.0.

b

Controls document length normalisation, typically 0.75.

$|D| / \text{avgdl}$

Length relative to the average; long documents are penalised.

Why it endures

No training, exact matching of names and codes, fast inverted indexes.

Worked example: scoring one document with BM25

Quantity	Term “refund”	Term “policy”
Documents containing term, n (of N = 1,000)	10	300
IDF = $\ln(1 + (N \text{ minus } n + 0.5) / (n + 0.5))$	$\ln(95.33) = 4.557$	$\ln(3.331) = 1.203$
Term frequency in document D, f	2	3
Saturated tf = $f \times 2.2 / (f + 1.2)$, with D = avgdl	$4.4 / 3.2 = 1.375$	$6.6 / 4.2 = 1.571$
Contribution = IDF $\times$ saturated tf	6.266	1.891

8.16

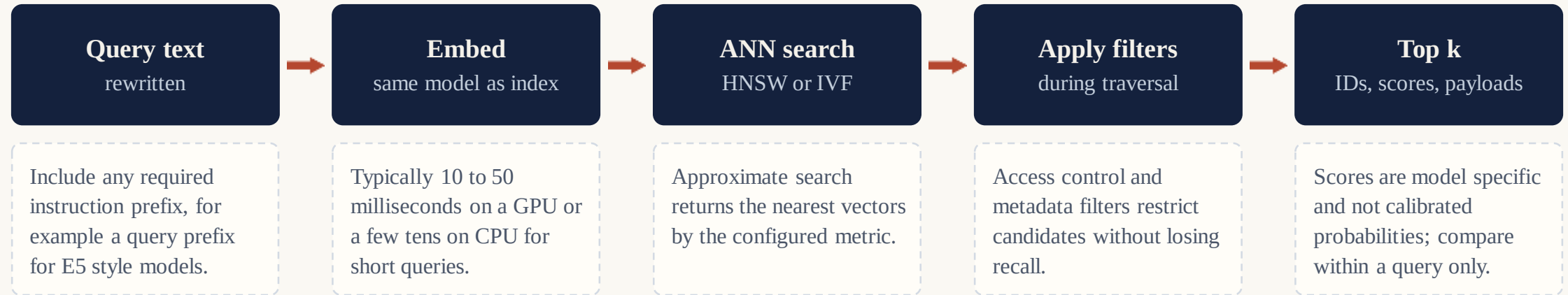
BM25 score of D for the query “refund policy”, with $k1 = 1.2$ and $b = 0.75$

6.45

Same document if twice the average length: saturated tf falls to 1.073 and 1.294

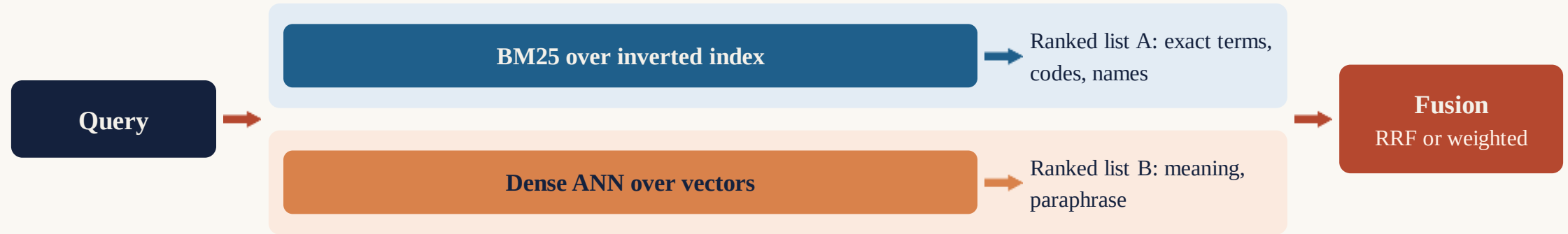
COMPONENT 7

Dense retrieval, step by step



COMPONENT 7

Hybrid retrieval architecture



Why it works

The two retrievers fail on different queries, so their union has higher recall than either.

Execution

Both searches run in parallel; latency is the slower of the two plus fusion.

Tuning

Weight the lists by query type: identifiers favour BM25, natural language favours dense.

COMPONENT 7

Reciprocal rank fusion, with a worked example

$$\text{RRF}(d) = \sum \text{ over ranked lists } r \text{ of } 1 / (k + \text{rank}_r(d)), \text{ with } k = 60$$

Chunk	BM25 rank	Dense rank	RRF score	Fused rank
B	2	1	$1/62 + 1/61 = 0.03252$	1
A	1	3	$1/61 + 1/63 = 0.03227$	2
D	absent	2	$1/62 = 0.01613$	3
C	3	absent	$1/63 = 0.01587$	4

Choosing k and similarity thresholds



First stage k

Large enough for recall, typically 50 to 200 when a reranker follows, or 5 to 20 without one. Plot recall at k to find where it plateaus.



Final k

What reaches the prompt, typically 3 to 10. Bounded by the context budget and by noise tolerance of the generator.



Score thresholds

Useful to drop clearly irrelevant results, but set per model and validate; relative cutoffs, such as within a margin of the top score, are more robust.

COMPONENT 7

Advanced retrieval techniques



Multi vector indexing

Index several vectors per chunk: the text, a summary, and hypothetical questions, all pointing to the same chunk.



Parent document

Match on small children, return the parent section. See the small to big pattern in Component 3.



ColBERT late interaction

Token level vectors with MaxSim scoring for fine grained matching.



Recency and authority boosts

Blend similarity with freshness or source quality scores.



Metadata aware retrieval

Self query filters extracted from the question, as in Component 6.

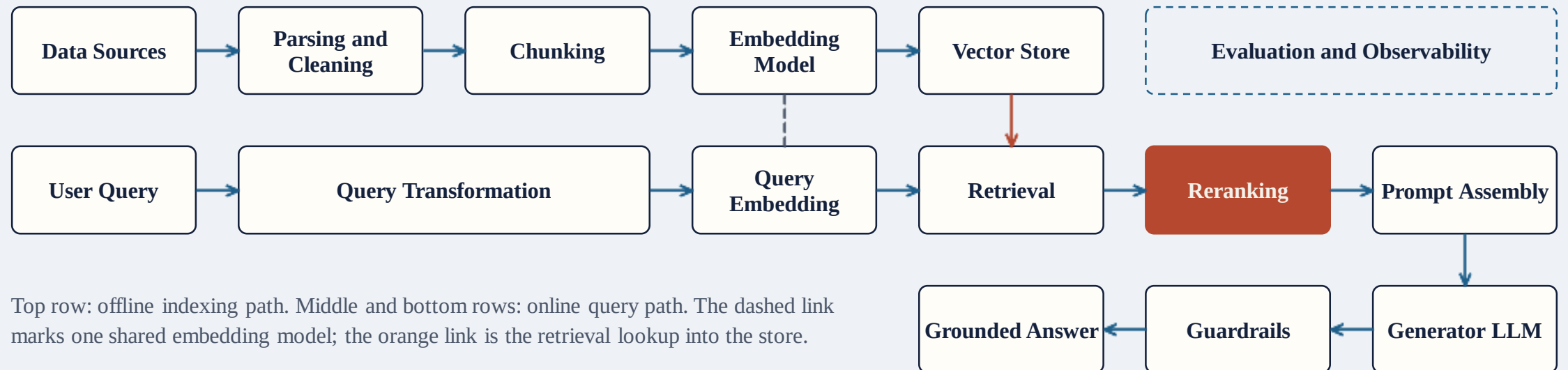


Iterative retrieval

Retrieve, read, reformulate and retrieve again for multihop questions.

Reranking and Post Retrieval

Rescoring, diversifying, compressing and ordering candidates so that the generator receives the smallest, strongest possible evidence set.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

Why retrieval has two stages

1,000,000

cross encoder pairs to score if applied to a one million chunk corpus: infeasible per query

100

pairs to score when the first stage hands over its top 100 candidates

10,000×

fewer expensive model calls, the reason retrieval is split into two stages

First stage: optimised for recall

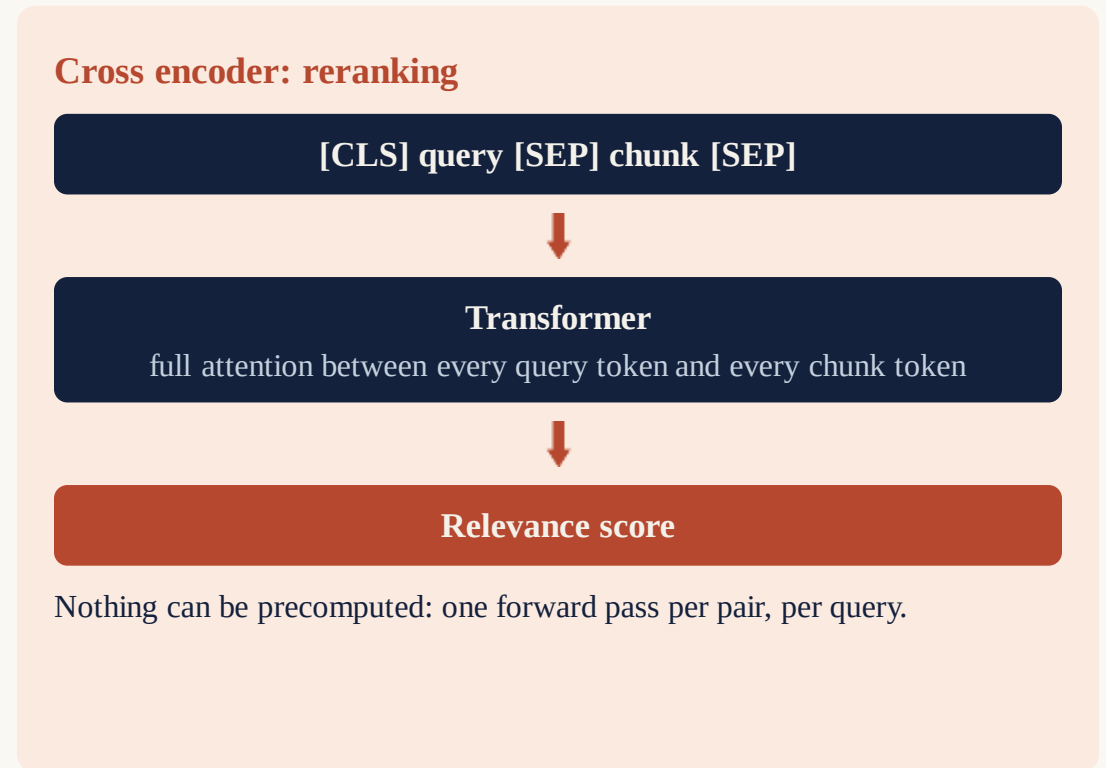
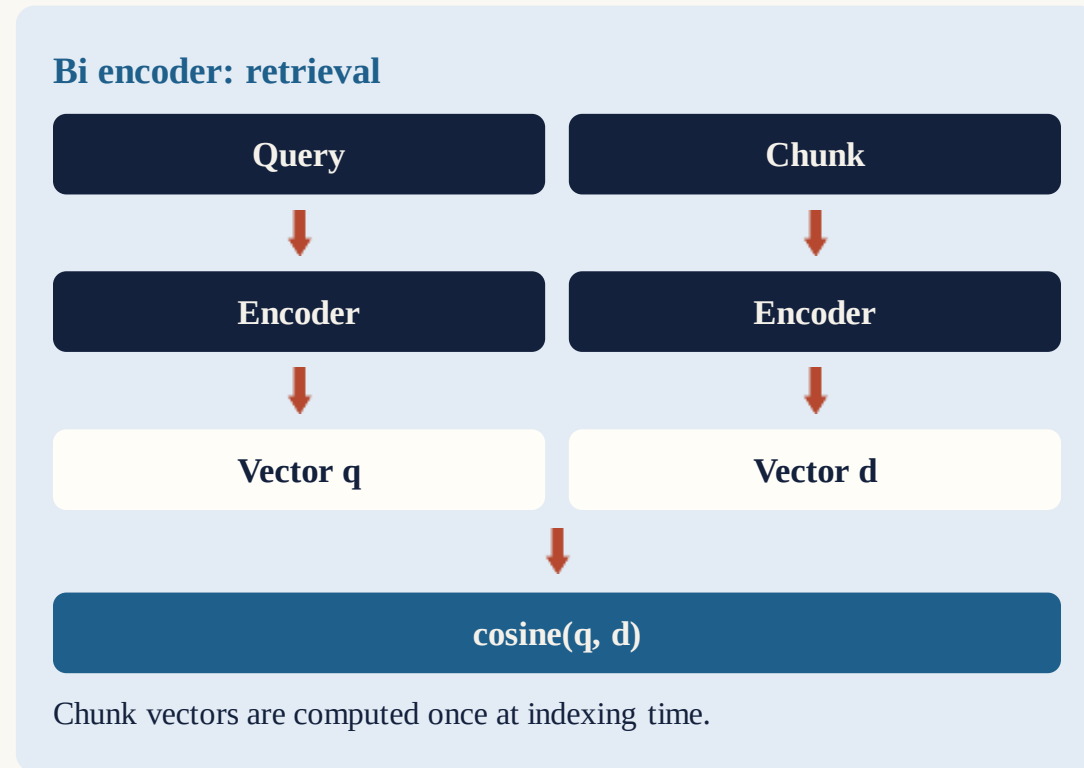
- Cheap scorers: BM25 and bi encoder vectors.
- Precomputed document representations.
- Goal: the answer is somewhere in the top 100.

Second stage: optimised for precision

- Expensive scorer: a cross encoder reading query and chunk together.
- Computed per query and candidate pair.
- Goal: the answer is in the top 3 to 5.

COMPONENT 8

Inside a cross encoder reranker



COMPONENT 8

Reranker options

Reranker type	Examples	Strength	Trade off
Cross encoder	MS MARCO MiniLM cross encoders, BGE rerankers	High accuracy, open weights, self hostable	Adds tens to hundreds of milliseconds
Hosted rerank API	Cohere Rerank, Voyage and Jina rerankers	Strong quality, no infrastructure	Network latency, data leaves your environment
Late interaction	ColBERTv2	Fast, token level matching	Larger index storage
LLM listwise	RankGPT style prompting of a general LLM	Very strong reasoning about relevance	Slow and costly per query
Heuristic	Recency, authority, business rules	Transparent, domain aware	Weak on semantic relevance alone

Diversity with maximal marginal relevance, worked

MMR = argmax over d of [$\lambda \cdot \text{Sim}(d, q)$ minus $(1 \text{ minus } \lambda) \cdot \text{max over s in S of } \text{Sim}(d, s)$]

Step	Candidate	Relevance to q	Max similarity to selected	MMR with $\lambda = 0.7$	Decision
1	c1	0.90	none selected yet, 0	0.630	Select c1
2	c2 (near duplicate of c1)	0.88	0.95	0.616 minus 0.285 = 0.331	Skip
2	c3 (different aspect)	0.80	0.30	0.560 minus 0.090 = 0.470	Select c3

Takeaway
Pure relevance ranking would pick c2, repeating c1. MMR (Carbonell and Goldstein, 1998) picks c3, adding new information to the context.

COMPONENT 8

Context compression and filtering



Extractive compression

Keep only sentences relevant to the query, selected by a small model or embedding similarity.



Abstractive compression

An LLM summarises each chunk with respect to the question before generation.



Token level pruning

Methods such as LLMingua drop low information tokens to shrink prompts.



Relevance filtering

Drop chunks a grader judges irrelevant, as in corrective RAG.



Deduplication

Remove repeated overlap spans and near duplicate chunks.



Neighbour expansion

The opposite move: add adjacent chunks when a match needs context.

COMPONENT 8

Ordering evidence in the prompt

Edge placement: the strongest evidence is placed at the beginning and the end of the context block, the weakest in the middle.



Prompt position: start of context on the left, end of context on the right.



Descending order

Simplest: best first. Works well when k is small after reranking.



Edge placement

Counters the lost in the middle effect for larger k, as shown above.

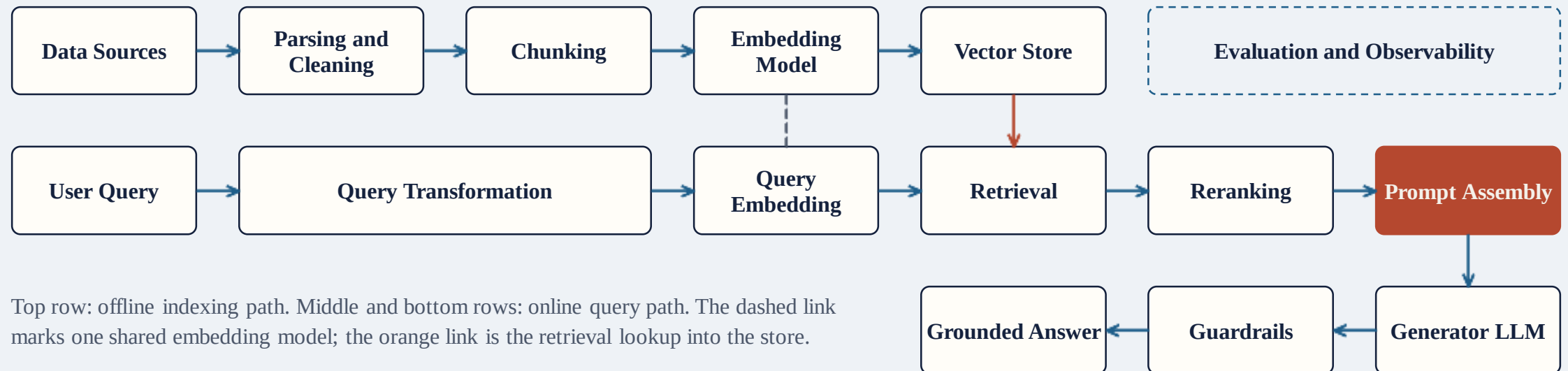


Document order

Keep chunks from one source in their original order to preserve narrative flow.

Prompt Assembly

Engineering the context: templates, instructions, evidence formatting, citation markers, token budgets and conversation memory.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

Anatomy of a RAG prompt



COMPONENT 9

A prompt template, annotated

EXAMPLE GROUNDED ANSWER TEMPLATE

You answer questions using only the documents provided.

If the documents do not contain the answer, say so plainly.

Cite sources with their IDs in square brackets after each claim.

<documents>

<doc id="HR4.2" title="Family leave" updated="March 2026"> ... </doc>

<doc id="CH7" title="Contractor Handbook"> ... </doc>

</documents>

Question: {question}

Answer in at most 150 words, with citations.

Delimiters

XML style tags separate trusted instructions from untrusted retrieved text.

Source attributes

IDs, titles and dates let the model cite and prefer recent sources.

Abstention

An explicit permission to say “not found” reduces invented answers.

COMPONENT 9

Token budgeting, worked

An 8,192 token window, allocated deliberately:



Block	Tokens	Share	Rule
System prompt	400	4.9%	Fixed, cacheable
Conversation history	1,000	12.2%	Summarise beyond the last few turns
Question	60	0.7%	Always included in full
Retrieved context	4,000	48.8%	Fill by rank until the budget is reached
Reserved output	1,024	12.5%	Must be reserved up front
Headroom	1,708	20.8%	Absorbs tokenizer variance and long chunks

COMPONENT 9

Citations and grounding

Making citations reliable

- Give every chunk a short, stable ID in the prompt.
- Require a citation after each factual sentence.
- Map IDs back to source URIs and page numbers in the interface.
- Validate that every cited ID was actually in the context.

Verifying that citations support claims

- Split the answer into claims with their citations.
- Check each claim against its cited chunk with an entailment model or LLM judge.
- Flag or remove unsupported sentences before display.
- Track citation precision and recall as quality metrics.

COMPONENT 9

Handling missing, conflicting and partial evidence



When evidence is missing

Instruct the model to state that the documents do not contain the answer, and to say what it did find. Offer a next step, such as contacting a team, instead of guessing.



When sources conflict

Surface both, with dates and authority: “the 2026 policy says X; the 2024 handbook says Y”. Prefer the newest authoritative source when rules say so.

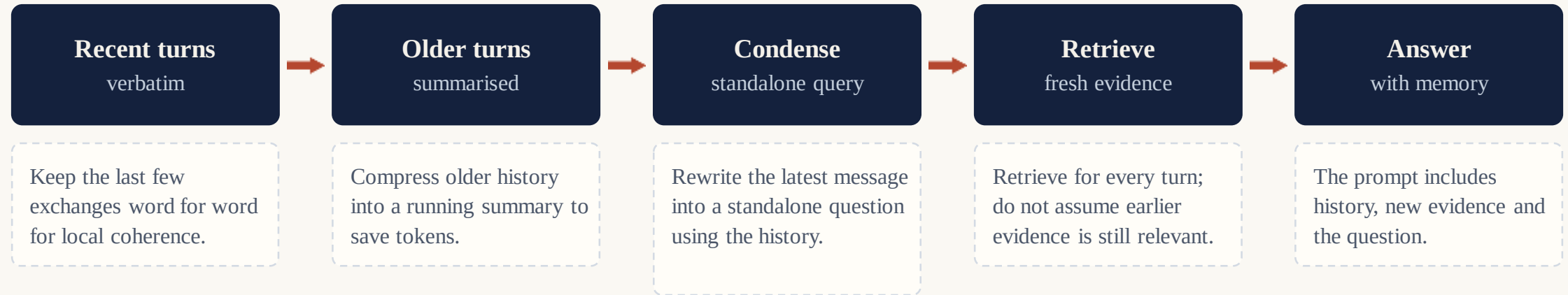


When evidence is partial

Answer the supported part and explicitly name the unsupported part, rather than completing it from general knowledge.

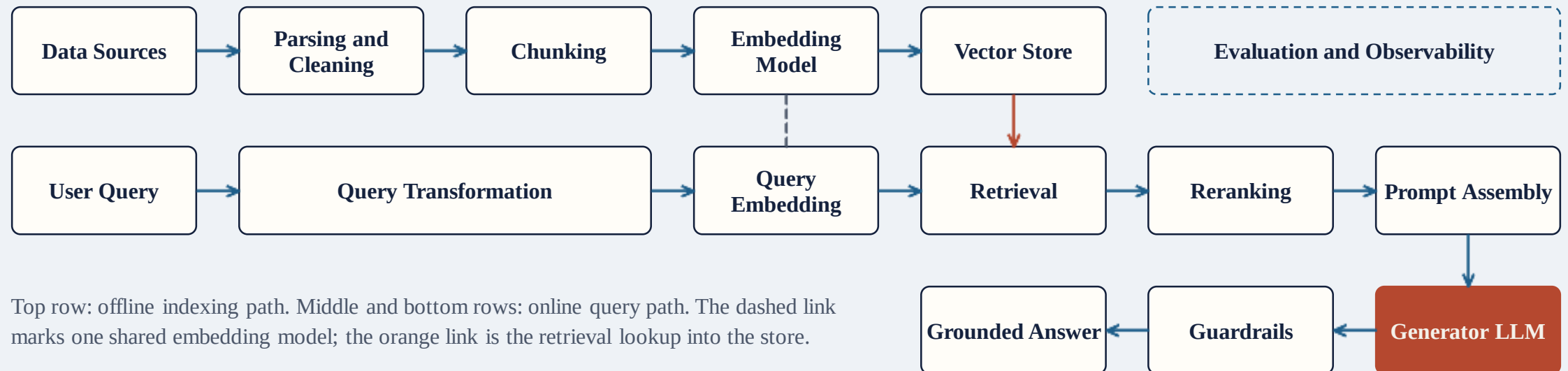
COMPONENT 9

Conversation memory in RAG



The Generator LLM

The model that reads the augmented prompt and writes the grounded answer: selection, decoding, structured output, adaptation and speed.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

COMPONENT 10

Selecting the generator

Criterion	Why it matters in RAG	How to assess
Faithfulness to context	Must prefer retrieved evidence over its own prior	Faithfulness score on your golden set
Instruction following	Citation formats, abstention, output schemas	Format compliance rate
Context length and use	Must read many chunks without losing the middle	Tests with the answer at varied positions
Latency and throughput	Dominates end to end response time	Time to first token, tokens per second
Cost	Multiplied by every query, with long prompts	Cost per answered query
Deployment constraints	Data residency, privacy, on premise needs	Hosting options and terms

Decoding parameters

Parameter	Effect	Recommended for grounded QA
Temperature	Randomness of token sampling	Low, around 0 to 0.3, for factual answers
Top p	Samples only from the most probable tokens up to mass p	Leave at default when temperature is low
Max output tokens	Hard cap on answer length	Set explicitly; also reserve it in the budget
Stop sequences	Ends generation at chosen markers	Useful for structured and templated outputs
Seed	Makes sampling repeatable where supported	Fix it for evaluation runs

Structured outputs and tool use



JSON schemas

Request answers as structured objects, for example answer, citations, confidence. Many APIs can enforce a schema, which removes parsing failures.



Function calling

Let the model call retrieval, SQL or calculators as tools, the foundation of agentic RAG covered in Chapter 4.



Extraction tasks

Use RAG to find the right documents, then a schema to extract fields such as dates, parties and amounts.

Adapting the generator to RAG

RAFT: Retrieval Augmented Fine Tuning

- Proposed by Zhang et al., 2024, at UC Berkeley.
- Train on question, documents, answer triples that include distractor documents.
- Some examples omit the relevant document entirely, teaching robustness.
- Answers are chain of thought with quotes from the evidence.

Other adaptation options

- Fine tune for your citation and output format only.
- Distil a large model's grounded answers into a smaller model.
- Preference tuning to reward abstention when evidence is missing.
- Train a Self RAG style model that critiques its own retrieval.

Latency and cost optimisations for generation



Stream tokens

Show the answer as it is generated; perceived latency becomes time to first token.



Prompt caching

Reuse the processed stable prefix, such as system instructions, to cut latency and cost.



Semantic caching

Return a stored answer when a new query is very similar to an earlier one.



Right size the model

Route simple questions to small models; reserve large models for hard ones.



Speculative decoding

A small draft model proposes tokens that the large model verifies in parallel.

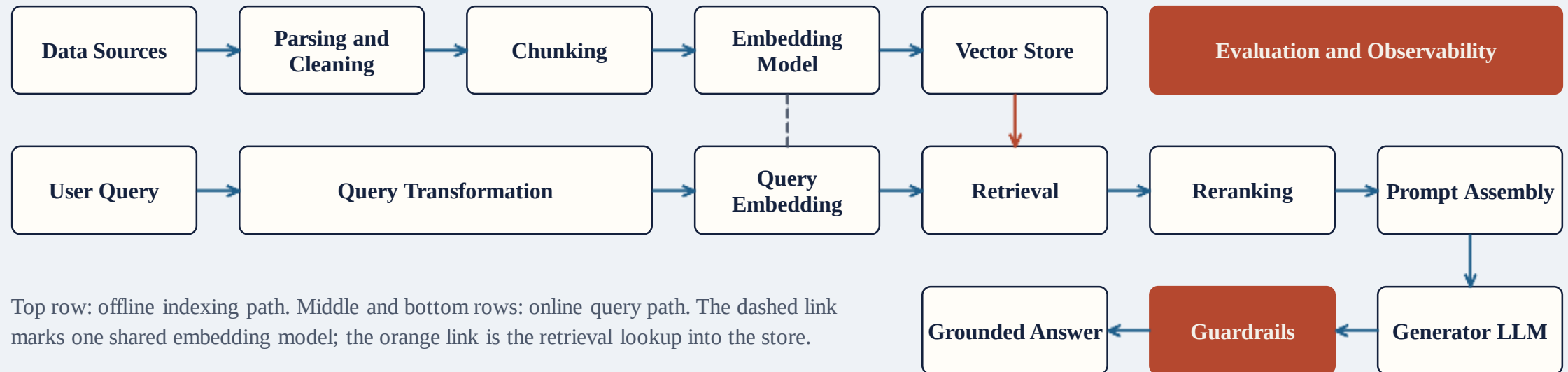


Shorter prompts

Fewer, better chunks after reranking and compression reduce prefill time.

Guardrails, Caching and Orchestration

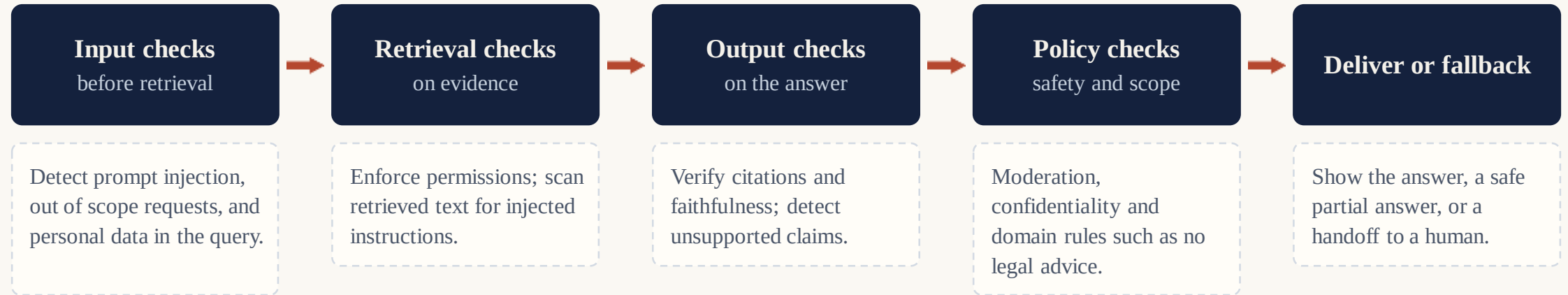
The cross cutting layer: verifying outputs, caching at every stage, coordinating the pipeline, and learning from production.



Top row: offline indexing path. Middle and bottom rows: online query path. The dashed link marks one shared embedding model; the orange link is the retrieval lookup into the store.

COMPONENT 11

The guardrail pipeline



COMPONENT 11

Caching at every layer

Exact answer cache

Key: normalised query plus user permissions

← Instant repeat answers; must respect access control

Semantic answer cache

Key: query embedding within a similarity threshold

← Catches paraphrases; risk of wrong matches

Retrieval result cache

Key: rewritten query plus filters

← Skips search and reranking on hot queries

Embedding cache

Key: content hash plus model version

← Avoids reembedding unchanged text

Prompt prefix cache

Provider side cache of stable prompt prefixes

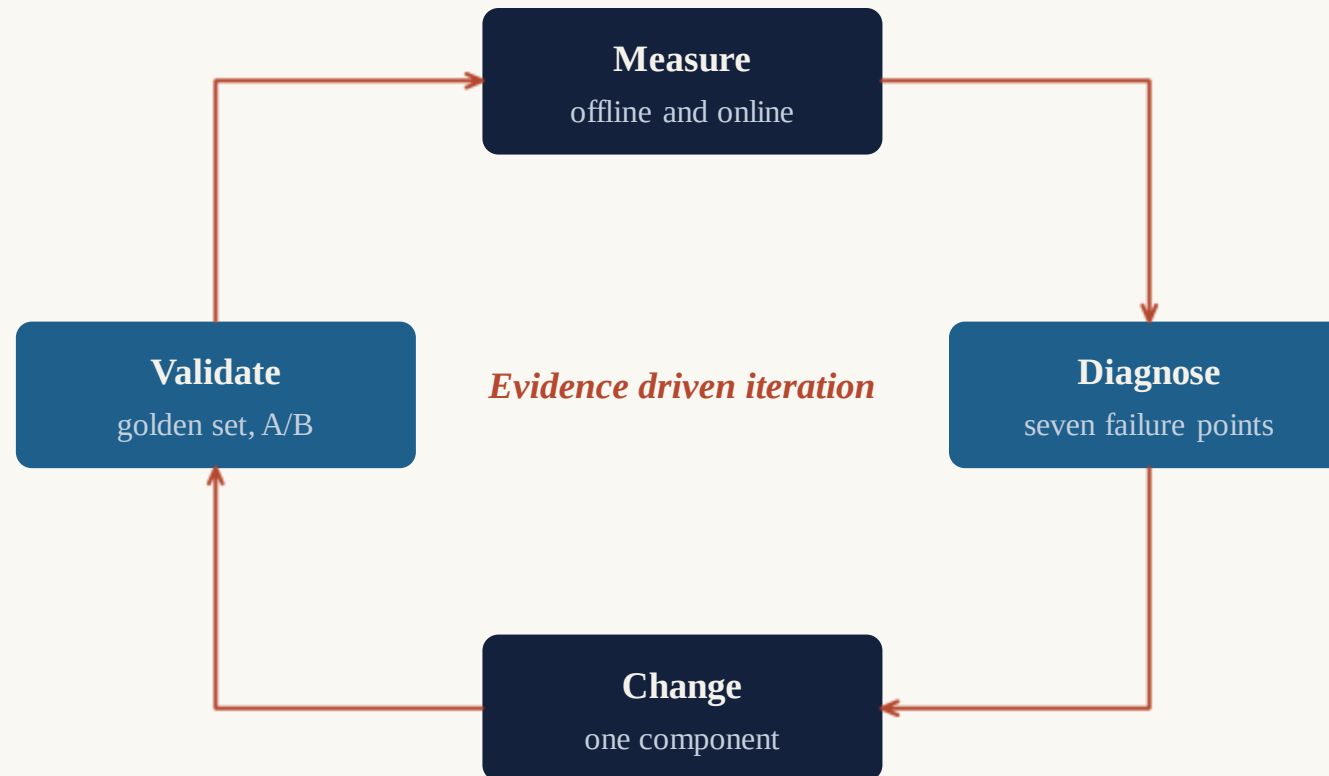
← Cuts generation latency and input cost

COMPONENT 11

Orchestration frameworks

Framework	Emphasis	Good fit
LangChain and LangGraph	Broad integrations; graph based agent workflows	Agentic and multi step pipelines
LlamaIndex	Data ingestion, indexing and retrieval abstractions	Document heavy RAG, advanced retrieval
Haystack	Explicit, typed pipelines of components	Production search and QA pipelines
DSPy	Programs whose prompts are optimised automatically	Systematic prompt and pipeline optimisation
Custom code	Direct calls to models and stores	Simple pipelines, maximum control

The continuous improvement loop



One change at a time

Change a single component per experiment so gains and regressions are attributable.

Regression gates

Block releases that lower recall, faithfulness or latency targets.

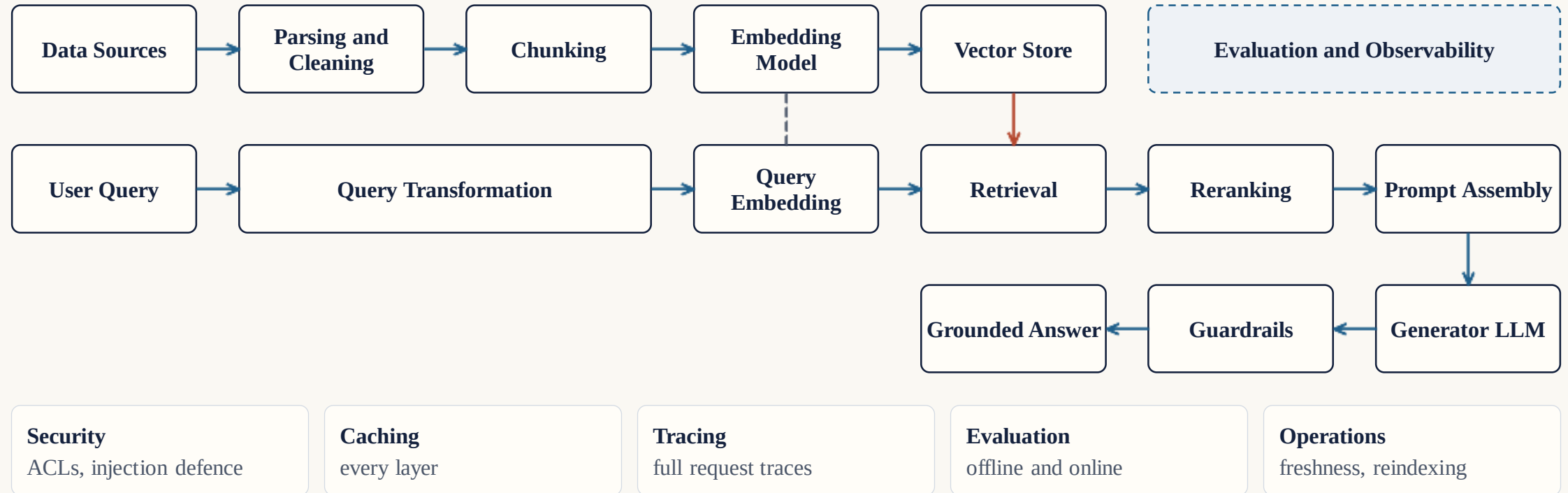
Grow the test set

Every production failure becomes a permanent test case.

SYNTHESIS

The production reference architecture

All eleven components, joined by the cross cutting services that make them production grade.



SYNTHESIS

From zero to production: a phased build plan

1

Week 1: Baseline

Ingest a representative sample, recursive chunking, one embedding model, top k dense retrieval, a simple grounded prompt.

3

Week 3: Retrieval quality

Improve parsing and chunking, add hybrid search and a reranker; tune k.

5

Week 5: Production readiness

ACL filtering, injection defences, tracing, caching, incremental ingestion.

2

Week 2: Measure

Build a golden set of 100 or more queries; compute recall at k, faithfulness and answer relevance.

4

Week 4: Generation quality

Citations, abstention, token budgeting, conflict handling; verify citations.

6

Week 6 onwards: Iterate

Query transformation, advanced retrieval or agentic patterns only where evaluation shows need.

CLOSING

Key takeaways

1. Answer quality is bounded by retrieval quality.
2. Parsing and chunking decide what can ever be found.
3. Hybrid retrieval plus reranking is the reliable core.
4. Ground, cite, verify, and permit abstention.
5. Measure every component; change one thing at a time.

FURTHER READING

Lewis et al. (2020) Retrieval Augmented Generation for Knowledge Intensive NLP Tasks

Karpukhin et al. (2020) Dense Passage Retrieval for Open Domain QA

Gao et al. (2023) Retrieval Augmented Generation for LLMs: A Survey

Liu et al. (2023) Lost in the Middle

Es et al. (2023) RAGAS: Automated Evaluation of RAG

Barnett et al. (2024) Seven Failure Points of RAG Systems

Malkov and Yashunin (2016) HNSW